Algorithmic Verification of Combinatorial Flips in Integer Sequences

Submitted September 2025, in partial fulfillment of
the conditions for the award of the degree **MSc Data Science.**

Ayush Saxena
20645113

Supervised by Prof. Hamid Abban

School of Computer Science
University of Nottingham

I hereby declare that this dissertation is all my own work, except as indicated in the text:

Signature: _____

Date: 12 / 09 / 25

I hereby declare that I have all necessary rights and consents to publicly distribute this dissertation via the University of Nottingham's e-dissertation archive.

Abstract

This dissertation investigates the problem of algorithmically verifying combinatorial flips in integer sequences, drawing on concepts from group theory, ring theory, and symbolic computation. The project formalises the flip verification problem by defining three necessary conditions based on greatest common divisors, positivity constraints, and modular inequalities that determine whether a given sequence represents a valid flip in three or higher dimensions.

A C++ implementation was developed to encode these conditions efficiently. Key algorithmic techniques such as backtracking for generating combinations, Euclidean methods for computing greatest common divisors, and quicksort for ordering coordinates were employed to ensure both correctness and computational feasibility. The methodology also includes a mathematical reformulation of the conditions, providing a clear bridge between the algebraic theory of flips and their computational verification.

Extensive experiments were carried out on a range of test cases, covering valid flips, invalid sequences, and edge cases where only subsets of the conditions are satisfied. The results confirmed that the program consistently distinguishes flips from non-flips, while also highlighting the relative difficulty of satisfying the modular inequality condition in higher dimensions. A complexity analysis demonstrated that although the implementation is polynomial in key components, the exponential nature of combination generation poses scalability challenges for very large inputs. Potential optimisations and directions for further research are discussed.

Overall, this work contributes both a theoretical framework and a practical algorithm for studying flips in integer sequences, offering a reproducible toolset that connects abstract algebraic structures with concrete computational verification.

Acknowledgements

I would like to express my deepest gratitude to my supervisor, **Dr. Hamid Abban**, for his continuous guidance, encouragement, and invaluable insights throughout the course of this dissertation. His expertise and constructive feedback were instrumental in shaping this work and keeping me focused in the right direction.

I am also thankful to the faculty and staff of the **School of Computer Science** and the **School of Mathematical Sciences** at the **University of Nottingham**, for providing the academic environment, resources, and support that enabled me to complete this research.

On a personal note, I would like to extend my heartfelt thanks to my family and friends for their unwavering support, patience, and encouragement during this journey. Their belief in me has been a constant source of motivation.

Finally, I would like to acknowledge all authors and researchers whose work has inspired and informed this dissertation. Any errors or shortcomings that remain are, of course, my own responsibility.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

The study of **flips** in algebraic geometry and commutative algebra occupies a central role in the broader framework of the Minimal Model Program (MMP), a research area that has shaped modern birational geometry since the work of Mori and others in the late twentieth century. Flips provide a mechanism for resolving singularities while preserving canonical class properties, and their verification lies at the intersection of deep structural theory and computational experimentation. Historically, algebraists have studied flips through the lens of graded rings, ideals, and quotient constructions [4], while geometers have emphasized their role in higher-dimensional classification. Yet the complexity of these structures motivates the use of algorithmic and symbolic techniques to test flip conditions concretely. This dissertation develops and evaluates such a computational framework, bridging the abstract theory of group and ring actions with explicit algorithmic verification.

1.1 Motivation

The motivation for this work arises from both theoretical and computational needs. On the theoretical side, flips remain a key step in extending the MMP beyond dimension three, where classical classification results no longer suffice. From a computational perspective, the verification of flips involves intricate number theoretic and algebraic conditions such as greatest common divisors, modular congruences, and group actions on integer sequences

that are well suited to symbolic and algorithmic treatment. Previous research in computer algebra and ideal membership has demonstrated the feasibility of translating algebraic properties into recursive procedures [3], providing a natural foundation for this project. The challenge addressed here is to design and implement an algorithm that can test whether a given sequence of integers encodes a valid flip, generalising from low dimensional examples to arbitrary n-dimensional cases.

1.2 Aims and Objectives

The overarching aim of this dissertation is to develop, implement, and evaluate an algorithmic approach for verifying combinatorial flips in integer sequences. This broad aim can be refined into the following objectives:

- To formalise the flip conditions in algebraic and combinatorial terms, providing precise mathematical formulations for arbitrary dimensions.
- To design and implement efficient algorithms that check these conditions using established computational techniques such as backtracking, modular arithmetic, and quicksort-based preprocessing.
- To evaluate the correctness and efficiency of the implementation through structured test cases, including both positive and negative examples.
- To analyse the computational complexity of the approach and identify avenues for optimisation and further research.

1.3 Description of the Work

This dissertation integrates abstract theory with practical computation. The early chapters provide the necessary algebraic background in group theory, ring theory, and symbolic computation, drawing especially on standard references such as Hungerford [4]. The methodology chapter details the algorithmic implementation, including the decomposition of flip conditions into explicit computational checks. A series of carefully chosen test

cases is then presented to illustrate the behaviour of the program, followed by a critical evaluation of results in terms of accuracy, time complexity, and robustness. The work culminates in a reflection on contributions and limitations, situating the algorithm within the wider landscape of algebraic geometry and computer algebra.

In summary, this dissertation tells a coherent story beginning from the historic motivation of flips in the MMP, grounding the discussion in group and ring theory, translating the abstract conditions into algorithms, and evaluating their performance in practice. This introduction sets the stage for the chapters that follow by connecting theoretical motivations with the computational contributions of the project.

Chapter 2

Background on Group Theory

This section presents the essential mathematical foundations required to understand the computational framework of flips in algebraic geometry. The aim is not to provide an exhaustive treatment of each concept, but rather to highlight and explain the structures and definitions most relevant to the combinatorial verification of flips. The material, including theorems, definitions, examples, and illustrative formulas, is primarily adapted from ***Algebra*** by ***Thomas W. Hungerford*** [4], which serves as a standard graduate level reference in abstract algebra. The topics covered include group theory, with a focus on finite and infinite groups and their actions, as well as the basics of algebraic geometry such as affine varieties and singularities. These concepts form the mathematical language and tools through which the behaviour of flips is formalized and studied. The purpose of this section is to provide a self contained yet sufficiently rigorous overview to support the algorithmic translation and implementation phases of the project.

2.1 Semigroups, Monoids and Groups

A **semigroup** is a nonempty set S equipped with a binary operation $\cdot$ such that the operation is associative, i.e., for all $a, b, c \in S$, we have $(a \cdot b) \cdot c = a \cdot (b \cdot c)$. In simple terms, a semigroup captures the idea of a collection of elements together with a rule for combining them, where the way in which elements are grouped does not affect the outcome. This minimal structure models many natural systems where composition is possible, such as

the addition of natural numbers or concatenation of strings, though it does not necessarily guarantee the presence of an identity element or inverses.

A **monoid** is a semigroup M that contains an identity element e, satisfying $e \cdot a = a \cdot e = a$ for all $a \in M$. Conceptually, this means that within the structure there exists a neutral element that does not alter other elements when combined with them. The identity provides a natural starting or "do-nothing" element, which is essential in contexts where operations need a baseline reference, such as multiplying by 1 or adding 0.

A **group** is a monoid G in which every element has an inverse. Formally, for each $a \in G$, there exists $a^{-1} \in G$ such that $a \cdot a^{-1} = a^{-1} \cdot a = e$. The presence of inverses allows any operation within the group to be undone, ensuring a form of symmetry and reversibility. Groups are fundamental in mathematics because they formalise the notion of symmetry and provide a framework for analyzing structures where operations can always be reversed. If, in addition, the operation is commutative—meaning $a \cdot b = b \cdot a$ for all $a, b \in G$—then the group is called an **abelian group**. Abelian groups describe systems where order does not matter in combining elements, such as integer addition or multiplication of nonzero real numbers.

These hierarchical structures semigroups, monoids, and groups represent successive layers of algebraic organisation, each adding an additional property that strengthens the structure. Understanding the distinctions and relationships between them provides the foundation for more advanced topics such as homomorphisms, cosets, and group actions.

Example 1: Semigroup of Natural Numbers under Addition Let $S = \mathbb{N} = \{1, 2, 3, \dots\}$ and define the binary operation $a \cdot b = a + b$. Then $(\mathbb{N}, +)$ is a semigroup:

- **Associativity:** For any $a, b, c \in \mathbb{N}$,

$$(a + b) + c = a + (b + c)$$

- **No identity:** There is no natural number e such that $e + a = a$ for all $a \in \mathbb{N}$, since

0 is not in $\mathbb{N}$ (if defined without 0).

Therefore, $(\mathbb{N}, +)$ is a semigroup, but not a monoid.

Example 2: Monoid of Strings under Concatenation Let S be the set of all finite strings (including the empty string ε) over some alphabet (e.g., English letters). Define the binary operation as string concatenation:

- **Associativity:** Concatenation is associative:

$$(\text{“}ab\text{”}+\text{“}cd\text{”})+\text{“}ef\text{”} = \text{“}abcd\text{”}+\text{“}ef\text{”} = \text{“}abcdef\text{”} = \text{“}ab\text{”}+(\text{“}cdef\text{”}) = \text{“}ab\text{”}+(\text{“}cd\text{”}+\text{“}ef\text{”})$$

- **Identity:** The empty string ε satisfies:

$$\varepsilon + s = s + \varepsilon = s \quad \text{for all strings } s$$

So, the set of strings with concatenation forms a monoid, but not a group because most strings do not have inverses under concatenation.

Example 3: Group of Integers under Addition Let $G = \mathbb{Z}$ and define the binary operation $a \cdot b = a + b$. Then $(\mathbb{Z}, +)$ is a group:

- **Associativity:** $(a + b) + c = a + (b + c)$ for all $a, b, c \in \mathbb{Z}$.
- **Identity:** 0 is the identity: $a + 0 = 0 + a = a$.
- **Inverses:** For each $a \in \mathbb{Z}$, the inverse is $-a$ because $a + (-a) = 0$.
- **Commutativity:** $a + b = b + a$ for all $a, b \in \mathbb{Z}$, so the group is abelian.

Example 4: Non-Abelian Group of 2×2 Invertible Matrices Let $G = GL_2(\mathbb{R})$, the set of all 2×2 invertible matrices with real entries, under matrix multiplication.

- Matrix multiplication is associative.

- The identity matrix $I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ acts as the identity.
- Every invertible matrix has an inverse such that $AA^{-1} = A^{-1}A = I$.
- **Not commutative:** In general, $AB \neq BA$ for matrices A, B.

Hence, $GL_2(\mathbb{R})$ is a non-abelian group.

Example 5: Multiplicative Group of Non-Zero Real Numbers Let $G = \mathbb{R}^* = \mathbb{R} \setminus \{0\}$ under multiplication.

- Associative: $(ab)c = a(bc)$ for all $a, b, c \in \mathbb{R}^*$.
- Identity: 1 is the identity element.
- Inverses: For each $a \in \mathbb{R}^*$, the inverse is $1/a$.
- Commutative: $ab = ba$ for all $a, b \in \mathbb{R}^*$.

So $(\mathbb{R}^*, \cdot)$ is an abelian group.

These foundational structures—semigroups, monoids, and groups—form the essential building blocks for all subsequent concepts in group theory. They will serve as the basis for understanding how transformations, symmetries, and algebraic structures interact in the broader framework of this dissertation.

2.2 Homomorphisms and Subgroups

A **group homomorphism** is a function $\varphi : G \to H$ between two groups $(G, \cdot)$ and $(H, *)$ such that for all $a, b \in G$,

$$\varphi(a \cdot b) = \varphi(a) * \varphi(b).$$

This means that the group operation is preserved under the mapping: combining two elements in G and then applying φ gives the same result as first applying φ to each element and then combining them in H. In simple terms, a homomorphism is a structure-

preserving map that translates the operation of one group consistently into another, ensuring that the essential algebraic structure is maintained.

A **subgroup** of a group G is a nonempty subset $H \subseteq G$ that is itself a group under the same operation as G. Formally, H must be closed under the group operation, contain the identity element of G, and include the inverse of every element it contains. Put differently, a subgroup inherits the group structure from G but is confined to a smaller collection of elements. Subgroups are important because they allow us to study smaller, well-structured pieces of a group while still working within the same overall framework. The concepts of homomorphisms and subgroups are deeply interconnected: subgroups represent internal structure within a group, while homomorphisms describe external structure-preserving relationships between groups. Together, they form the backbone of group theory by providing the language to analyse both the internal composition of groups and their external interactions with one another.

Example 1: Homomorphism from $\mathbb{Z}$ to $\mathbb{Z}_n$ Let $\phi : \mathbb{Z} \to \mathbb{Z}_n$ be defined by $\phi(k) = k \mod n$.

- **Homomorphism:** For any $a, b \in \mathbb{Z}$:

$$\phi(a+b) = (a+b) \mod n = (a \mod n + b \mod n) \mod n = \phi(a) + \phi(b)$$

- **Kernel:** $\ker \phi = \{k \in \mathbb{Z} \mid k \equiv 0 \mod n\} = n\mathbb{Z}$.

- **Image:** $\operatorname{Im} \phi = \mathbb{Z}_n$ (surjective).

This shows a basic but powerful homomorphism that reduces infinite structure to a finite one.

Example 2: Homomorphism Between Matrix Groups Let $G = GL_2(\mathbb{R})$, the group of all invertible 2×2 real matrices, and let $\phi : G \to \mathbb{R}^*$ be the determinant map: $\phi(A) = \det(A)$.

- **Homomorphism:** $\det(AB) = \det(A) \cdot \det(B)$.
- **Kernel:** $\ker \phi = SL_2(\mathbb{R})$ — the special linear group of matrices with determinant 1.
- **Image:** $\mathbb{R}^* = \mathbb{R} \setminus \{0\}$.

This example shows how structural properties like invertibility and scaling relate through homomorphisms.

Example 3: Subgroup of Even Integers Let $G = \mathbb{Z}$ and $H = 2\mathbb{Z} = \{..., -4, -2, 0, 2, 4, ...\}$.

- Identity: $0 \in H$.
- Closure: Sum of two even numbers is even.
- Inverses: Negative of an even number is even.

Hence, H is a subgroup of $\mathbb{Z}$.

Example 4: Nontrivial Subgroup of $\mathbb{Z}_6$ Let $G = \mathbb{Z}_6 = \{0, 1, 2, 3, 4, 5\}$ under addition mod 6, and let $H = \{0, 3\}$.

- $0 + 3 = 3$, $3 + 3 = 6 \equiv 0 \mod 6$.
- Identity: $0 \in H$.
- Closure and inverses: 3 is its own inverse modulo 6.

Thus, H is a subgroup of $\mathbb{Z}_6$.

Example 5: Trivial and Whole Group Subgroups Every group G has two guaranteed subgroups:

- The trivial subgroup $\{e\}$.
- The group G itself.

These are useful when reasoning about extreme cases or testing subgroup criteria.
Group homomorphisms and subgroups are essential for analyzing and comparing group structures. They allow us to map complexity into simpler components, detect structural similarities, and define key constructions like kernels, images, and quotient groups that will appear repeatedly throughout this dissertation.

2.3 Cyclic Groups

A **cyclic group** is a group G that can be generated by a single element $g \in G$, meaning every element of G can be written as a power of g. Formally,

$$G = \langle g \rangle = \{g^n \mid n \in \mathbb{Z}\}.$$

If G is finite, then there exists some smallest positive integer m such that $g^m = e$, the identity of G, and in this case $|G| = m$. If G is infinite, then all distinct powers of g produce different elements, and G is isomorphic to the group of integers under addition.
In plain terms, a cyclic group is one where a single element generates the entire structure by repeated application of the group operation and its inverse. For finite groups, this means going around in cycles until eventually returning to the identity, while for infinite groups it means continuing indefinitely without repetition. The element g is called a **generator** of the group, and a group may have one or more such generators depending on its structure.
Cyclic groups are fundamental because they provide the simplest nontrivial examples of groups and form building blocks for more complex algebraic systems. They also exhibit a direct connection to number theory, since the structure of cyclic groups is closely tied to the properties of integers and modular arithmetic.

Example 1: The Infinite Cyclic Group $(\mathbb{Z}, +)$ Let $G = \mathbb{Z}$ under addition.

- Define $g = 1$. Then the set of all integer multiples of 1 is:

$$\langle 1 \rangle = \{..., -3, -2, -1, 0, 1, 2, 3, ...\} = \mathbb{Z}$$

- Every integer can be written as $n \cdot 1$ for some $n \in \mathbb{Z}$.
- The generator 1 (or -1) produces all elements of the group.

Hence, $(\mathbb{Z}, +)$ is an infinite cyclic group generated by 1.

Example 2: The Finite Cyclic Group $\mathbb{Z}_6$ Let $G = \mathbb{Z}_6 = \{0, 1, 2, 3, 4, 5\}$ under addition modulo 6.

- Consider $g = 1$. Then:

$$\langle 1 \rangle = \{1, 1+1, 1+1+1, 1+1+1+1, \dots\} \mod 6 = \{1, 2, 3, 4, 5, 0\} = \mathbb{Z}_6$$

  So 1 is a generator of the group.

- Now try $g = 2$:

$$\langle 2 \rangle = \{2, 4, 0, 2, 4, 0, \dots\} = \{0, 2, 4\}$$

  This is a proper subgroup of $\mathbb{Z}_6$, not the whole group. Hence, 2 is not a generator.

- Similarly:

$$\langle 3 \rangle = \{3, 0, 3, 0, \dots\} = \{0, 3\}$$

  Only 1 and 5 are generators of $\mathbb{Z}_6$.

This example shows how a finite cyclic group has multiple possible generators and how each generator cycles through the elements in a specific order.

Example 3: Subgroups of a Cyclic Group Let us explore the subgroups of $\mathbb{Z}_{12}$.

- $\mathbb{Z}_{12}$ is cyclic because it can be generated by 1:

$$\langle 1 \rangle = \{0, 1, 2, ..., 11\}$$

- Every divisor d of 12 corresponds to a unique subgroup of order d:

$$\text{Subgroups: } \langle 12/d \rangle$$

- The subgroups of $\mathbb{Z}_{12}$ are:

$$\langle 0 \rangle = \{0\} \quad \text{(order 1)}$$
$$\langle 6 \rangle = \{0, 6\} \quad \text{(order 2)}$$
$$\langle 4 \rangle = \{0, 4, 8\} \quad \text{(order 4)}$$
$$\langle 3 \rangle = \{0, 3, 6, 9\} \quad \text{(order 4)}$$
$$\langle 2 \rangle = \{0, 2, 4, 6, 8, 10\} \quad \text{(order 6)}$$
$$\langle 1 \rangle = \mathbb{Z}_{12} \quad \text{(order 12)}$$

- Each subgroup is itself cyclic.

This example illustrates that the subgroup structure of a cyclic group is completely determined by the divisors of the group order.

Example 4: Multiplicative Group $\mathbb{Z}_7^*$ Consider the group $\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$ under multiplication modulo 7.

- Check if 3 is a generator:

$$3^1 = 3, \quad 3^2 = 2, \quad 3^3 = 6, \quad 3^4 = 4, \quad 3^5 = 5, \quad 3^6 = 1$$

- The powers of 3 modulo 7 generate the entire group.

So $\mathbb{Z}_7^*$ is a cyclic group, and 3 is a generator.

Cyclic groups are key examples in group theory, representing the most basic but complete group structure. They serve as essential building blocks in algebraic classification, modu-

lar arithmetic, and abstract symmetry, making them highly relevant for both theoretical study and practical applications.

2.4 Cosets and Lagrange's Theorem

Given a group G and a subgroup $H \leq G$, a **left coset** of H in G with respect to an element $g \in G$ is defined as

$$gH = \{g \cdot h \mid h \in H\}.$$

Similarly, a **right coset** is defined as

$$Hg = \{h \cdot g \mid h \in H\}.$$

Cosets partition the group G into disjoint subsets, each of which has the same size as H. In other words, the group can be viewed as a collection of translated copies of its subgroup. Intuitively, cosets represent how a subgroup "sits" inside the larger group, with each coset being a shifted version of the subgroup.

Theorem 2.1 (**Lagrange's Theorem**). *If G is a finite group and H is a subgroup of G, then the order (number of elements) of H divides the order of G. Formally,*

$$|G| = |H| \cdot [G : H],$$

*where $[G : H]$ denotes the number of distinct cosets of H in G, called the **index** of H in G.*

In simpler terms, Lagrange's Theorem tells us that the size of a subgroup always divides evenly into the size of the whole group. This result is powerful because it imposes strict restrictions on the possible subgroup sizes and immediately rules out certain group structures. For example, a group of prime order cannot have any nontrivial subgroups, since its only divisors are 1 and the prime itself. The concepts of cosets and Lagrange's Theorem thus form a cornerstone of finite group theory, providing a bridge between subgroup

structure and the overall size of a group.

Example 1: Cosets in $\mathbb{Z}_6$ Let $G = \mathbb{Z}_6 = \{0, 1, 2, 3, 4, 5\}$ under addition modulo 6. Consider the subgroup $H = \{0, 3\}$.

- The order of G is $|G| = 6$, and $|H| = 2$.
- Compute the left cosets:

$$0 + H = \{0, 3\}$$
$$1 + H = \{1, 4\}$$
$$2 + H = \{2, 5\}$$

- All other cosets (e.g., $3 + H$) repeat an existing coset: $3 + H = \{3, 0\} = 0 + H$.
- So the distinct cosets are $\{0, 3\}, \{1, 4\}, \{2, 5\}$.

We see that there are three cosets, and $6 = 2 \times 3$, confirming Lagrange's theorem.

Example 2: Cosets in the Group S_3 Let $G = S_3$, the symmetric group on 3 elements:

$$S_3 = \{e, (12), (13), (23), (123), (132)\}$$

Let $H = \{e, (12)\}$.

- $|G| = 6$, $|H| = 2$.
- Compute left cosets by choosing representatives:

$$eH = \{e, (12)\}$$
$$(13)H = \{(13), (132)\}$$
$$(23)H = \{(23), (123)\}$$

- These three cosets partition S_3 into equal parts.

Again, we confirm that $[G : H] = 3$, and $6 = 2 \times 3$.

Example 3: Cosets in $\mathbb{Z}$ modulo $3\mathbb{Z}$ Let $G = \mathbb{Z}$ under addition, and $H = 3\mathbb{Z} = \{..., -6, -3, 0, 3, 6, 9, ...\}$.

- Cosets are of the form:

$$0+H = \{..., -6, -3, 0, 3, 6, ...\} 1+H = \{..., -5, -2, 1, 4, 7, ...\} 2+H = \{..., -4, -1, 2, 5, 8, ...\}$$

- These cosets partition $\mathbb{Z}$ into equivalence classes modulo 3.
- Each coset is infinite, but the number of distinct cosets is 3.

This is an example of cosets in an infinite group, but with a finite index. We write the quotient group as $\mathbb{Z}/3\mathbb{Z} = \{[0], [1], [2]\}$.

Example 4: Visual Interpretation with $\mathbb{Z}_{12}$ and Subgroup $\langle 4 \rangle$ Let $G = \mathbb{Z}_{12}$ and $H = \langle 4 \rangle = \{0, 4, 8\}$.

- $|G| = 12$, $|H| = 3$, so we expect $[G : H] = 4$ cosets.
- Compute cosets:

$$0 + H = \{0, 4, 8\}$$
$$1 + H = \{1, 5, 9\}$$
$$2 + H = \{2, 6, 10\}$$
$$3 + H = \{3, 7, 11\}$$

- These are all distinct cosets of size 3.

This example gives a clear sense of how cosets partition the group evenly and illustrates counting in a cyclic context.

Conclusion: Cosets are essential for understanding group partitions and counting arguments in algebra. They underlie several critical results like Lagrange's theorem, and form the basis for defining quotient groups and analyzing symmetry in a structured way.

2.5 Normal Subgroups and Quotient Groups

A **normal subgroup** of a group G is a subgroup $N \leq G$ such that for every $g \in G$,

$$gNg^{-1} = N,$$

or equivalently, $gng^{-1} \in N$ for all $n \in N$. In other words, a subgroup is normal if it is invariant under conjugation by elements of the group. Normality ensures that the subgroup is not distorted when "shifted" within the group, making it compatible with the group's internal symmetries. This property is fundamental because it allows us to construct new groups, called factor groups, in a consistent way.

Given a normal subgroup N of G, the **factor group** (or **quotient group**) G/N is defined as the set of left cosets of N in G with the operation

$$(gN)(hN) = (gh)N.$$

This operation is well defined precisely because N is normal; without normality, the product of cosets could depend on the choice of representatives. Intuitively, forming the factor group means collapsing all elements of N to a single identity-like element and then studying the group structure that emerges on the set of cosets.

Normal subgroups and factor groups are central to group theory because they provide a way of breaking down groups into simpler pieces. By passing to quotients, one can study complex groups through their simpler "building blocks." This process is closely connected to homomorphisms: the kernel of a group homomorphism is always a normal subgroup, and the First Isomorphism Theorem shows how factor groups naturally arise from this

relationship.

Example 1: Normal Subgroup in $\mathbb{Z}$ and the Quotient Group $\mathbb{Z}/n\mathbb{Z}$ Let $G = \mathbb{Z}$ and $N = n\mathbb{Z} = \{..., -2n, -n, 0, n, 2n, ...\}$.

- $\mathbb{Z}$ is abelian, so every subgroup is normal.
- Cosets of N in $\mathbb{Z}$ are:

$$0 + N, 1 + N, 2 + N, \ldots, (n-1) + N$$

- These are equivalence classes modulo n.
- The quotient group $\mathbb{Z}/n\mathbb{Z}$ has n elements:

$$\mathbb{Z}/n\mathbb{Z} = \{[0], [1], [2], ..., [n-1]\}$$

This is the classical construction of modular arithmetic. The group operation is addition modulo n.

Example 2: Kernel of a Homomorphism from $GL_2(\mathbb{R})$ to $\mathbb{R}^*$ Let $G = GL_2(\mathbb{R})$ (group of invertible 2×2 matrices), and define $\phi : G \to \mathbb{R}^*$ by $\phi(A) = \det(A)$.

- ϕ is a group homomorphism since $\det(AB) = \det(A) \cdot \det(B)$.
- The kernel is:

$$\ker \phi = \{A \in GL_2(\mathbb{R}) \mid \det(A) = 1\} = SL_2(\mathbb{R})$$

- Since it's a kernel, $SL_2(\mathbb{R})$ is a normal subgroup of $GL_2(\mathbb{R})$.
- The quotient group $GL_2(\mathbb{R})/SL_2(\mathbb{R})$ captures how matrices differ by scaling.

This quotient is important in geometry and representation theory.

Example 3: Normal Subgroup in S_3 and Its Quotient

Let $G = S_3 = \{e, (12), (13), (23), (123), (132)\}$and let $N = A_3 = \{e, (123), (132)\}$.

- A_3 is the alternating group of even permutations.
- A_3 is a subgroup of S_3 of order 3.
- Conjugation preserves cycle type in S_3, so A_3 is normal: $gAg^{-1} \subseteq A_3$.
- The cosets of A_3 in S_3 are:

$$eA_3 = \{e, (123), (132)\}$$
$$(12)A_3 = \{(12), (13), (23)\}$$

- The quotient group S_3/A_3 has 2 elements and is isomorphic to $\mathbb{Z}_2$.

This example shows how a non-abelian group like S_3 can have a normal subgroup with an abelian quotient.

Conclusion: Normal subgroups and quotient groups are powerful tools in abstract algebra. They simplify group structures and provide insight into how different parts of a group relate. Homomorphisms naturally lead to quotient constructions, and many classification results in group theory rely on understanding the interaction between a group, its normal subgroups, and their quotient groups.

2.6 Finitely Generated Abelian Groups

An abelian group G is said to be **finitely generated** if there exists a finite set of elements $g_1, g_2, \ldots, g_n \in G$ such that every element of G can be expressed as a linear combination of these generators under the group operation. Formally,

$$G = \langle g_1, g_2, \ldots, g_n \rangle = \{\, a_1 g_1 + a_2 g_2 + \cdots + a_n g_n \mid a_i \in \mathbb{Z} \,\}.$$

This means that the entire group can be built up from finitely many elements, with all members of G obtained by combining these generators with integer coefficients.

The **Structure Theorem for Finitely Generated Abelian Groups** provides a complete classification of such groups. It states that every finitely generated abelian group G is isomorphic to a direct sum of the form

$$G \cong \mathbb{Z}^r \oplus \mathbb{Z}_{n_1} \oplus \mathbb{Z}_{n_2} \oplus \cdots \oplus \mathbb{Z}_{n_k},$$

where $r \geq 0$, each $n_i > 1$, and the divisibility condition $n_1 \mid n_2 \mid \cdots \mid n_k$ holds.

In plain terms, this theorem shows that every finitely generated abelian group can be decomposed into two parts: a **free part**, represented by $\mathbb{Z}^r$, and a **torsion part**, represented by the finite cyclic groups $\mathbb{Z}_{n_i}$. The free part consists of elements of infinite order, while the torsion part contains all elements of finite order. Importantly, this decomposition is unique up to isomorphism, meaning that although the choice of generators may vary, the structure of the group itself is fixed.

Finitely generated abelian groups are therefore among the most well-understood structures in algebra. Their classification not only provides a clear picture of their internal composition but also serves as a foundation for more advanced topics in group theory and its applications.

Example 1: $\mathbb{Z} \oplus \mathbb{Z}_4$ Let $G = \mathbb{Z} \oplus \mathbb{Z}_4$. This group consists of ordered pairs (a, b) where $a \in \mathbb{Z}$ and $b \in \mathbb{Z}_4 = \{0, 1, 2, 3\}$.

- The group operation is component-wise:

$$(a_1, b_1) + (a_2, b_2) = (a_1 + a_2, (b_1 + b_2) \mod 4)$$

- G is generated by $(1, 0)$ and $(0, 1)$:

$$\langle (1, 0), (0, 1) \rangle = \{(a, b) \mid a \in \mathbb{Z}, b \in \mathbb{Z}_4\}$$

- The group is infinite due to the $\mathbb{Z}$ component.

This example has a free rank of 1 and a torsion part of order 4. It fits the structure theorem with $r = 1$, $n_1 = 4$.

Example 2: $\mathbb{Z}_2 \oplus \mathbb{Z}_4$ Let $G = \mathbb{Z}_2 \oplus \mathbb{Z}_4$, consisting of pairs (a, b) where $a \in \{0, 1\}$ and $b \in \{0, 1, 2, 3\}$.

- Group operation: $(a_1, b_1) + (a_2, b_2) = ((a_1 + a_2) \mod 2, (b_1 + b_2) \mod 4)$.
- All elements have finite order.
- The orders of some elements:

$$\text{ord}(0, 1) = 4$$
$$\text{ord}(1, 0) = 2$$
$$\text{ord}(1, 2) = \text{lcm}(2, 2) = 2$$

- This is a finite abelian group of order $2 \times 4 = 8$.

Although this group has 8 elements, it is **not** cyclic because no single element generates the entire group. It decomposes into a direct sum of two cyclic groups.

Example 3: $\mathbb{Z}^2$ Let $G = \mathbb{Z}^2 = \mathbb{Z} \oplus \mathbb{Z}$, the set of all ordered pairs of integers (a, b).

- Operation: $(a, b) + (c, d) = (a + c, b + d)$.
- G is generated by $(1, 0)$ and $(0, 1)$.
- Every element can be written as $a(1, 0) + b(0, 1) = (a, b)$.
- All elements have infinite order.

This is a purely free abelian group with no torsion. According to the structure theorem, $G \cong \mathbb{Z}^2$ with $r = 2$, and $k = 0$ (no torsion components).

Example 4: Classification of Finite Abelian Groups of Order 8 Let's classify all abelian groups of order 8 up to isomorphism. Since $8 = 2^3$, the classification involves partitioning the exponent 3:

- $\mathbb{Z}_8$
- $\mathbb{Z}_4 \oplus \mathbb{Z}_2$
- $\mathbb{Z}_2 \oplus \mathbb{Z}_2 \oplus \mathbb{Z}_2$

Each of these is a distinct abelian group structure. This example illustrates how the structure theorem can be used to enumerate all possibilities for finite abelian groups.

Conclusion: Finitely generated abelian groups provide a rich but well-understood class of algebraic structures. The structure theorem allows us to decompose these groups into combinations of free and finite cyclic parts, offering a clear view into their algebraic nature. This clarity makes them particularly useful in a wide variety of mathematical and applied contexts, including number theory, module theory, and symmetry analysis.

2.7 The Action of a Group on a Set

A **group action** is a way for a group G to operate on a set X in a manner consistent with the group structure. Formally, an action of G on X is a function

$$G \times X \to X, \quad (g, x) \mapsto g \cdot x$$

such that for all $g, h \in G$ and $x \in X$:

- Identity: $e \cdot x = x$, where e is the identity element of G.
- Compatibility: $(gh) \cdot x = g \cdot (h \cdot x)$.

These conditions ensure that group elements behave like transformations of X, acting coherently with the group structure.
Associated with a group action are two important concepts:

- The **orbit** of an element $x \in X$ is

$$\text{Orb}(x) = \{g \cdot x \mid g \in G\},$$

which describes the set of points in X that x can be moved to under the action of G.

- The **stabilizer** of $x \in X$ is

$$\text{Stab}(x) = \{g \in G \mid g \cdot x = x\},$$

which describes the subgroup of G consisting of all elements that leave x unchanged.

In simple terms, group actions capture how the symmetries encoded by a group can be applied to a set. Orbits represent how far a point can travel under the group's influence, while stabilizers describe the symmetries that fix that point. This interplay between movement and symmetry makes group actions fundamental across algebra, geometry, and combinatorics.

Example 1 (Simple Action): Let $G = \mathbb{Z}_2 = \{0, 1\}$ act on the set $X = \{a, b\}$ by defining $0 \cdot x = x$ (identity action) and $1 \cdot a = b$, $1 \cdot b = a$. This simply swaps the two elements. The orbit of a is $\{a, b\}$ and its stabilizer is $\{0\}$. This is the simplest nontrivial example of a group action, showing how a group of size two can permute a set of two elements.

Example 2 (Algebraic Action with Numbers): Let $G = \mathbb{Z}$ act on the set $X = \mathbb{Z}_n = \{0, 1, \ldots, n-1\}$ by addition modulo n, defined as $k \cdot x = (x + k) \bmod n$. This action corresponds to rotating elements around a circle of size n. For instance, if $n = 5$, the orbit of 0 is all of $\mathbb{Z}_5$, and the stabilizer of each element is the trivial subgroup $\{0\}$. This algebraic example illustrates how infinite groups like $\mathbb{Z}$ can act naturally on finite cyclic sets.

Example 3 (Geometric Action): Let $G = D_4$, the dihedral group of order 8 (sym-

metries of a square), act on the set X of vertices of the square. The action is defined by applying each symmetry (rotations and reflections) to the vertices. For example, a 90° rotation cycles the vertices, while a reflection swaps pairs of opposite corners. The orbit of any vertex is the full set of four vertices, showing that the group acts transitively. The stabilizer of a vertex consists of the symmetries that fix that corner, namely the identity and the reflection through the diagonal containing that vertex. This geometric example demonstrates how group actions formalize familiar notions of symmetry.

Conclusion: Group actions provide a powerful framework for linking abstract group structure to concrete situations. They explain how groups can move or preserve elements of a set, and the notions of orbits and stabilizers offer key insights into the structure of these actions. The examples above—from a simple two-element swap, to modular arithmetic, to geometric symmetries—highlight the versatility of group actions across different mathematical domains. These ideas play a central role in deeper results such as the Orbit-Stabilizer Theorem and Burnside's Lemma.

Chapter 3

Background on Ring Theory

This section presents the essential mathematical foundations required to understand the computational framework of flips in algebraic geometry. The aim is not to provide an exhaustive treatment of each concept, but rather to highlight and explain the structures and definitions most relevant to the combinatorial verification of flips. The material, including theorems, definitions, examples, and illustrative formulas, is primarily adapted from ***Algebra*** by ***Thomas W. Hungerford*** [4], which serves as a standard graduate level reference in abstract algebra. The topics covered include group theory, with a focus on finite and infinite groups and their actions, as well as the basics of algebraic geometry such as affine varieties and singularities. These concepts form the mathematical language and tools through which the behaviour of flips is formalized and studied. The purpose of this section is to provide a self contained yet sufficiently rigorous overview to support the algorithmic translation and implementation phases of the project.

3.1 Rings and Homomorphisms

A **ring** is a nonempty set R equipped with two binary operations, usually denoted by addition $(+)$ and multiplication $(\cdot)$, that satisfy the following conditions:

1. $(R, +)$ is an abelian group.

2. Multiplication is associative, i.e., $(ab)c = a(bc)$ for all $a, b, c \in R$.

3. The distributive laws hold: $a(b+c) = ab+ac$ and $(a+b)c = ac+bc$ for all $a, b, c \in R$.

If multiplication is commutative, i.e., $ab = ba$ for all $a, b \in R$, then R is called a **commutative ring**. Furthermore, if there exists an element $1_R \in R$ such that $1_R \cdot a = a \cdot 1_R = a$ for all $a \in R$, then R is called a **ring with identity**.

A **ring homomorphism** between two rings R and S is a function $f : R \to S$ such that for all $a, b \in R$,

$$f(a + b) = f(a) + f(b) \quad \text{and} \quad f(ab) = f(a)f(b).$$

The **kernel** of a ring homomorphism is

$$\ker f = \{a \in R \mid f(a) = 0_S\},$$

while the **image** of f is

$$\mathrm{Im} f = \{f(a) \mid a \in R\} \subseteq S.$$

In plain terms, rings extend the concept of groups by incorporating both addition and multiplication. The additive structure must form an abelian group, while multiplication only needs to be associative and distribute over addition. This makes rings versatile algebraic structures that serve as the backbone of many areas of mathematics, from number theory and algebraic geometry to functional analysis. A ring homomorphism is a structure-preserving map that respects both operations, allowing us to translate problems from one ring into another while preserving their algebraic essence. The kernel of a homomorphism identifies elements that collapse to zero under the mapping, capturing structural information about the source ring, while the image describes the portion of the target ring that is reached. Together, these ideas form the foundation for studying the internal structure of rings and their relationships to one another.

Example 1: The Integers and Modular Arithmetic Let $R = \mathbb{Z}$ and $S = \mathbb{Z}_n$ for some fixed $n \in \mathbb{N}$. Define the function $\phi : \mathbb{Z} \to \mathbb{Z}_n$ by

$$\phi(k) = k \mod n.$$

- ϕ is a ring homomorphism: it preserves addition and multiplication.
- $\phi(a+b) = (a+b) \mod n = (a \mod n + b \mod n) \mod n = \phi(a) + \phi(b)$.
- $\phi(ab) = (ab) \mod n = (a \mod n \cdot b \mod n) \mod n = \phi(a)\phi(b)$.
- The kernel is $\ker \phi = \{k \in \mathbb{Z} \mid k \equiv 0 \mod n\} = n\mathbb{Z}$.
- The image is all of $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$.

This homomorphism maps integers into the finite ring of integers modulo n.

Example 2: Ring of 2×2 Matrices Let $R = M_2(\mathbb{R})$, the ring of 2×2 real matrices. This ring is:

- Non-commutative: matrix multiplication does not satisfy $AB = BA$ in general.
- Has identity: the identity matrix $I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$.
- Not a field: many matrices are not invertible.

Now define the map $f : M_2(\mathbb{R}) \to \mathbb{R}$ by $f(A) = \operatorname{tr}(A)$, where $\operatorname{tr}(A)$ is the trace of the matrix.

- This is not a ring homomorphism, because $f(AB) \neq f(A)f(B)$ in general.

This example helps illustrate that not every familiar function between rings is a ring homomorphism.

Example 3: Polynomial Rings Let $R = \mathbb{R}$ and consider $R[x]$, the ring of all polynomials with real coefficients.

Define a homomorphism $f : \mathbb{R}[x] \to \mathbb{R}$ by evaluation at 1, i.e., $f(p(x)) = p(1)$.

- For polynomials $p(x), q(x) \in \mathbb{R}[x]$, we have:

$$f(p(x) + q(x)) = p(1) + q(1) = f(p(x)) + f(q(x)),$$

$$f(p(x)q(x)) = p(1)q(1) = f(p(x))f(q(x)).$$

- Hence, f is a ring homomorphism.

- $\ker f = \{p(x) \in \mathbb{R}[x] \mid p(1) = 0\}$.

This kind of homomorphism — called an evaluation homomorphism — plays an important role in algebraic geometry and functional analysis.

Example 4: Endomorphism Ring of an Abelian Group Let A be an abelian group and define $\mathrm{End}(A)$ to be the set of group homomorphisms $f : A \to A$.

- Define addition: $(f + g)(a) = f(a) + g(a)$.

- Define multiplication: $(f \circ g)(a) = f(g(a))$ (composition).

- Then $\mathrm{End}(A)$ becomes a ring, often noncommutative.

- Identity: the identity map id_A acts as multiplicative identity.

This example shows how structure-preserving functions themselves can form rings.

Example 5: Group Ring Let G be a finite group and R a ring (say $\mathbb{R}$). Then the group ring $R(G)$ consists of formal sums

$$\sum_{g \in G} r_g g, \quad \text{with } r_g \in R.$$

Addition and multiplication are defined naturally via the ring R and the group G.

- These formal sums behave like polynomials in the group elements.

- Group rings play a fundamental role in representation theory and module theory.

These examples demonstrate the richness and flexibility of the concept of rings and ring homomorphisms. Understanding them is essential for analyzing the algebraic structure underlying a wide variety of mathematical objects.

3.2 Ideals

An **ideal** of a ring R is a nonempty subset $I \subseteq R$ with two key properties. First, $(I, +)$ is a subgroup of $(R, +)$, meaning it is closed under subtraction: if $a, b \in I$, then $a - b \in I$. Second, I must absorb multiplication by elements of R. More precisely, if $a \in I$ and $r \in R$, then $ra \in I$ for a left ideal, and $ar \in I$ for a right ideal. If both conditions hold, I is called a two-sided ideal, which in commutative rings is simply referred to as an ideal.
A special case is a **principal ideal**. In a ring R, an ideal I is principal if it is generated by a single element $a \in R$, that is,

$$I = (a) = \{ra \mid r \in R\}.$$

Principal ideals show how entire subsets of a ring can be generated from repeated multiplication of a single element, reflecting the role of generators in building algebraic structure. Conceptually, ideals play for rings the role that normal subgroups play for groups. They capture subsets that are compatible with the ring structure in such a way that we can form meaningful quotients. In commutative rings, the distinction between left and right ideals disappears, simplifying the theory. Ideals are central to generalizing divisibility, defining prime and maximal elements, and building quotient rings. They also form the foundation of advanced topics such as modules and algebraic geometry, where ideals correspond to algebraic varieties. In this way, ideals serve as the key tool for transferring the algebraic structure of a ring into deeper theoretical and geometric contexts.

Example 1: Even Integers as an Ideal in $\mathbb{Z}$ Let $R = \mathbb{Z}$ and define $I = 2\mathbb{Z} = \{2n \mid n \in \mathbb{Z}\}$.

- I is an additive subgroup of $\mathbb{Z}$.

- For any $a \in \mathbb{Z}$ and $2n \in I$, $a \cdot 2n = 2(an) \in I$.
- Since $\mathbb{Z}$ is commutative, this is both a left and right ideal.
- This is also a principal ideal: $I = (2)$.

So $2\mathbb{Z}$ is an ideal in $\mathbb{Z}$ and represents all multiples of 2.

Example 2: Ideal of Matrices with Zero in a Column Let $R = M_2(\mathbb{R})$, the ring of 2×2 matrices with real entries. Define:

$$I = \left\{ \begin{bmatrix} 0 & b \\ 0 & d \end{bmatrix} \mid b, d \in \mathbb{R} \right\}$$

- I is a left ideal:
  - Closed under subtraction.
  - For any $A \in M_2(\mathbb{R})$ and $B \in I$, the product AB remains in I.
- I is not a right ideal: BA may not preserve the zero column.

This is an example of a one-sided ideal, only satisfying one direction of multiplication closure.

Example 3: Principal Ideals in $\mathbb{Z}[x]$ Let $R = \mathbb{Z}[x]$, the ring of polynomials with integer coefficients. Define the ideal $I = (x^2 + 1) = \{f(x)(x^2 + 1) \mid f(x) \in \mathbb{Z}[x]\}$.

- I is the set of all multiples of the polynomial $x^2 + 1$.
- It is an ideal because:
  - Closed under addition and subtraction.
  - Closed under multiplication by any polynomial in $\mathbb{Z}[x]$.
- It is a principal ideal generated by $x^2 + 1$.

This example demonstrates how ideals generalize the idea of "divisibility" in polynomial rings.

Example 4: Zero and Unit Ideals In any ring R:

- The set $\{0\}$ is always an ideal (called the zero ideal).
- The whole ring R is also an ideal (called the unit ideal or improper ideal).

These form the minimal and maximal elements in the lattice of ideals.

Conclusion: Ideals are fundamental tools in ring theory, enabling quotient constructions and facilitating factorization, decomposition, and classification. Principal ideals provide a bridge between familiar number-theoretic concepts and abstract algebra, while left and right ideals reveal structural asymmetries in non-commutative rings.

3.3 Factorization in Commutative Rings

A **commutative ring** with identity $1 \neq 0$ is called an **integral domain** if it has no zero divisors, meaning that whenever $ab = 0$ for $a, b \in R$, then either $a = 0$ or $b = 0$. Integral domains provide a natural setting for studying divisibility, as they ensure that multiplication behaves in a predictable way.

Within an integral domain, several important classes of elements can be distinguished. An element $u \in R$ is called a **unit** if there exists $v \in R$ such that $uv = 1$. Units represent the invertible elements of the ring. An element $a \in R$, not a unit and not zero, is called **irreducible** if whenever $a = bc$, then either b or c is a unit. Irreducibles are the ring-theoretic analogue of prime numbers in the integers. An element $p \in R$, again not a unit and not zero, is called **prime** if $p \mid ab$ implies $p \mid a$ or $p \mid b$. Every prime element is irreducible, but not every irreducible element is necessarily prime unless the ring has additional properties.

A central concept in factorization theory is that of a **unique factorization domain (UFD)**. An integral domain R is a UFD if every nonzero, non-unit element of R can be written as a product of irreducibles, and this factorization is unique up to reordering and multiplication by units. This generalises the familiar fundamental theorem of arithmetic for the integers, which guarantees unique factorization into prime numbers.

In plain terms, factorization in commutative rings extends the familiar process of breaking integers into primes. The notions of units, irreducibles, and primes allow us to formalise how elements of a ring can be decomposed. However, not every integral domain has unique factorization. Some rings admit different irreducible decompositions of the same element that are not equivalent, which is why the UFD property is significant. In a UFD, we recover the familiar arithmetic property that factorization is unique, ensuring that irreducible and prime elements coincide. This makes UFDs a cornerstone of algebra, bridging the study of number theory with the more general framework of ring theory.

Example 1: Factorization in $\mathbb{Z}$ The ring of integers $\mathbb{Z}$ is a classic example of a UFD.

- Units: ± 1
- Irreducibles = Primes: $2, 3, 5, 7, \ldots$
- Every integer can be factored uniquely into primes up to sign and order:

$$60 = 2^2 \cdot 3 \cdot 5 = (-3) \cdot (-2) \cdot 2 \cdot 5 \text{ (same up to units)}$$

Example 2: Irreducibles vs Primes in $\mathbb{Z}[\sqrt{-5}]$ Consider $R = \mathbb{Z}[\sqrt{-5}] = \{a + b\sqrt{-5} \mid a, b \in \mathbb{Z}\}$.

- Factor 6 in two different ways:

$$6 = 2 \cdot 3 = (1 + \sqrt{-5})(1 - \sqrt{-5})$$

- None of these factors are associates (not units times each other).
- So $\mathbb{Z}[\sqrt{-5}]$ is not a UFD.
- Here, irreducible elements like $1 + \sqrt{-5}$ are not prime.

This ring illustrates that unique factorization is not guaranteed in all integral domains.

Example 3: UFD of Polynomials $\mathbb{Q}[x]$ Let $R = \mathbb{Q}[x]$, the ring of polynomials with rational coefficients.

- This is a UFD.
- Every polynomial factors uniquely into irreducible polynomials over $\mathbb{Q}$.
- For example:

$$x^4 - 1 = (x^2 - 1)(x^2 + 1) = (x - 1)(x + 1)(x^2 + 1)$$

- The irreducibles here are $x - 1$, $x + 1$, and $x^2 + 1$.

Example 4: UFDs and Principal Ideal Domains (PIDs) Every principal ideal domain (PID) is a UFD, but not every UFD is a PID.

- $\mathbb{Z}$ is a PID $\Rightarrow$ also a UFD.
- $\mathbb{Z}[x]$ is a UFD but **not** a PID.

This shows that the notion of unique factorization can extend beyond Euclidean or principal rings.

Conclusion: Factorization in rings generalizes the fundamental theorem of arithmetic. Unique factorization domains are rings where every element can be "broken down" into irreducibles in a well-defined way. Understanding which rings support this property is essential in algebra, number theory, and polynomial analysis.

3.4 Rings of Quotients and Localization

Just as the field of rational numbers $\mathbb{Q}$ can be constructed from the integers $\mathbb{Z}$ by inverting all nonzero integers, more general rings can also be expanded by formally inverting certain elements. This process is known as **localization**. Localization allows us to enlarge a given ring R into a new ring where chosen elements become invertible, providing a powerful tool for analysing local behaviour within algebraic structures.

To formalise this, we first introduce the notion of a **multiplicative set**. A subset $S \subseteq R$ is called multiplicative if it contains 1 and is closed under multiplication, i.e., if $a, b \in S$, then $ab \in S$. This ensures that the set provides a consistent collection of denominators that can safely be inverted.

Given a ring R and a multiplicative set S, the **localization of R at S**, denoted $S^{-1}R$, is the ring consisting of fractions of the form

$$\frac{r}{s}, \quad r \in R,\ s \in S,$$

subject to the equivalence relation

$$\frac{r_1}{s_1} = \frac{r_2}{s_2} \quad \Longleftrightarrow \quad \exists t \in S \text{ such that } t(r_1 s_2 - r_2 s_1) = 0.$$

In the special case where $S = R \setminus P$ for a prime ideal P, the localization is written R_P and is referred to as the localization of R at P.

Conceptually, localization provides a way to "zoom in" on the structure of a ring around certain elements or ideals. By inverting specific elements, we create a new setting in which those elements behave like units, much as fractions extend the integers by making every nonzero integer invertible. This technique is especially important in algebraic geometry and number theory. For example, localizing at a prime ideal corresponds to studying the behaviour of functions near a given point on a variety. In this way, rings of quotients and localization offer a flexible method for focusing on local properties while preserving algebraic consistency.

Example 1: Field of Fractions $\mathbb{Q}$ from $\mathbb{Z}$ Let $R = \mathbb{Z}$ and $S = \mathbb{Z} \setminus \{0\}$. Then S is a multiplicative set.

- The localization $S^{-1}\mathbb{Z}$ is the field of fractions:

$$S^{-1}\mathbb{Z} = \left\{ \frac{a}{b} \mid a \in \mathbb{Z}, b \in \mathbb{Z} \setminus \{0\} \right\} = \mathbb{Q}.$$

- This is the most basic example of localization: making every nonzero integer invertible.

Example 2: Localization of $\mathbb{Z}$ at a Prime p Let $R = \mathbb{Z}$ and $S = \mathbb{Z} \setminus p\mathbb{Z}$, the set of integers not divisible by a fixed prime p.

- This S is a multiplicative set.
- The localization $\mathbb{Z}_{(p)} = S^{-1}\mathbb{Z}$ consists of rational numbers a/b where b is not divisible by p.
- $\mathbb{Z}_{(p)}$ is a local ring — it has a unique maximal ideal consisting of elements with numerator divisible by p.

This allows arithmetic that "ignores" divisibility by p, useful in number theory.

Example 3: Localization in $\mathbb{Z}[x]$ Let $R = \mathbb{Z}[x]$, and let S be the set of all polynomials in $\mathbb{Z}[x]$ whose constant term is not zero.

- S is multiplicatively closed since the constant term of a product is the product of constant terms.
- The localization $S^{-1}\mathbb{Z}[x]$ contains rational functions where denominators are polynomials with nonzero constant terms.
- This allows us to "invert" many polynomials without forming the full field of fractions.

This type of localization appears in formal geometry and when defining rings of formal Laurent series.

Example 4: Non-commutative Localization In general, localization in non-commutative rings is more subtle. Multiplicative sets must satisfy the Ore condition to ensure that the localization exists.

- For example, in rings of differential operators, localization can still be defined, but care must be taken with left/right ideals.

Conclusion: Localization is a powerful tool in commutative algebra, allowing algebraists to isolate and examine the local behavior of rings and modules. By selectively inverting elements, one constructs new rings that retain enough of the original structure while enabling deeper analysis of algebraic and arithmetic properties.

3.5 Rings of Polynomials and Formal Power Series

Given a commutative ring R, the **polynomial ring** in one variable over R, denoted $R[x]$, is the set of all finite expressions of the form

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n, \quad a_i \in R,$$

with addition and multiplication defined in the usual way by combining like terms and distributing products. Polynomial rings extend a ring by adjoining an indeterminate x, and they form the algebraic foundation for describing equations, curves, and higher-dimensional varieties.

The **formal power series ring** over R, denoted $R[[x]]$, generalises this construction by allowing infinitely many terms:

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots, \quad a_i \in R.$$

Unlike polynomials, elements of $R[[x]]$ need not terminate. Multiplication in $R[[x]]$ is defined formally by convolution:

$$\left(\sum_{i=0}^{\infty} a_ix^i\right) \cdot \left(\sum_{j=0}^{\infty} b_jx^j\right) = \sum_{k=0}^{\infty} \left(\sum_{i=0}^{k} a_ib_{k-i}\right) x^k.$$

Conceptually, polynomial rings $R[x]$ represent the most natural way to enlarge a ring R to include algebraic expressions involving a variable. They underpin much of algebra and geometry, since polynomial equations are used to define curves, surfaces, and varieties. On the other hand, formal power series rings $R[[x]]$ extend this idea further by permitting

infinite expansions. These are not functions in the analytic sense, but formal objects that are indispensable in combinatorics, algebraic geometry, and number theory. While polynomials capture finite algebraic behaviour, formal power series are powerful tools for representing infinite processes and structures within an algebraic framework.

Example 1: Polynomial Ring $\mathbb{Z}[x]$ Let $R = \mathbb{Z}$.

- The elements of $\mathbb{Z}[x]$ are finite sums of the form $a_0 + a_1x + \cdots + a_nx^n$ with $a_i \in \mathbb{Z}$.
- The ring is commutative with identity and is an integral domain.
- For example:

$$f(x) = 2x^2 + 3x + 1, \quad g(x) = x^3 - 4$$

$$f(x) + g(x) = x^3 + 2x^2 + 3x - 3,$$

$$f(x) \cdot g(x) = 2x^5 + 3x^4 - 8x^2 + 3x - 4.$$

$\mathbb{Z}[x]$ is not a field, but it is a UFD.

Example 2: Formal Power Series Ring $\mathbb{Q}[[x]]$ Let $R = \mathbb{Q}$.

- An example of a formal power series:

$$f(x) = 1 + x + x^2 + x^3 + x^4 + \dots$$

- This series does not terminate, so it is not a polynomial.
- Multiplication uses convolution:

$$(1 + x + x^2 + \dots)(1 - x) = 1,$$

  which shows $f(x) = \frac{1}{1-x}$ as a formal identity.

- In $\mathbb{Q}[[x]]$, we can divide by $1 - x$ because $1 - x$ has constant term 1, a unit in $\mathbb{Q}$.

Formal power series allow algebraic manipulation without concern for convergence.

Example 3: Applications in Combinatorics Formal power series are used to encode sequences.

- Let a_n be the number of ways to write n as a sum of 1s and 2s.
- The generating function is:

$$G(x) = \frac{1}{1 - x - x^2} = \sum_{n=0}^{\infty} a_n x^n.$$

- We compute terms recursively:

$$a_0 = 1, \quad a_1 = 1, \quad a_n = a_{n-1} + a_{n-2} \quad \text{(Fibonacci numbers)}.$$

- Thus, $G(x)$ encodes the Fibonacci sequence.

This showcases how $\mathbb{Z}[[x]]$ models infinite discrete structures.

Example 4: $R[x_1, x_2, \ldots, x_n]$ and Multivariate Rings Let $R = \mathbb{R}$ and consider $R[x, y]$.

- This is the ring of bivariate polynomials with real coefficients.
- Example:

$$f(x, y) = x^2 + 2xy + y^2, \quad g(x, y) = x - y$$

- Useful in algebraic geometry for defining curves and surfaces like conic sections.

Conclusion: Polynomial rings and formal power series form the backbone of modern algebra and discrete mathematics. Polynomials model finite algebraic expressions and structures, while power series capture infinite behavior and recurrence. Mastery of both is essential for deeper exploration in fields ranging from algebraic geometry to combinatorics and beyond.

3.6 Factorization in Polynomial Rings

Let R be an integral domain. The polynomial ring $R[x]$ consists of all finite-degree polynomials with coefficients in R, and just as with integers, one of the central questions is how these polynomials can be factored into simpler building blocks. A nonzero, non-unit polynomial $f(x) \in R[x]$ is called **irreducible over** R if it cannot be written as a product of two polynomials of strictly smaller degree with coefficients in R. Irreducible polynomials thus play the same role in polynomial rings as prime numbers do in the integers.

A key result in this context is that if R is a **unique factorization domain (UFD)**, then $R[x]$ is also a UFD. This means that every nonzero polynomial can be expressed as a product of irreducible polynomials, and this factorization is unique up to multiplication by units and reordering of factors. In particular, over a field, the polynomial ring $F[x]$ always has this property, making polynomial factorization a reliable and well-structured process.

Conceptually, factorization in polynomial rings generalises the familiar prime factorization of integers. Just as integers can be decomposed uniquely into prime numbers, polynomials over a field or a UFD can be decomposed uniquely into irreducible polynomials. This perspective is central to solving algebraic equations, constructing field extensions, and analysing roots. Importantly, irreducibility depends on the base ring: a polynomial that is irreducible over $\mathbb{Q}$ may factor further when considered over $\mathbb{R}$ or $\mathbb{C}$. Thus, the study of factorization in polynomial rings not only deepens our understanding of algebraic structure but also connects directly to broader themes in number theory and algebraic geometry.

Example 1: Factorization over $\mathbb{Q}$, $\mathbb{R}$, and $\mathbb{C}$ Let $f(x) = x^2 + 1$.

- Over $\mathbb{Q}$: irreducible (no rational roots).
- Over $\mathbb{R}$: irreducible (no real roots).
- Over $\mathbb{C}$:

$$f(x) = (x + i)(x - i)$$

so it factors completely.

This shows that irreducibility depends on the base field.

Example 2: Eisenstein's Criterion for Irreducibility Let $f(x) = x^4 + 4x^3 + 6x^2 + 8x + 10 \in \mathbb{Z}[x]$.

We apply Eisenstein's Criterion at prime $p = 2$:

- All coefficients except leading are divisible by 2.
- Constant term 10 is not divisible by $2^2 = 4$.

So by Eisenstein's Criterion, $f(x)$ is irreducible over $\mathbb{Q}$.

Example 3: Reducible Polynomial in $\mathbb{Z}[x]$ Let $f(x) = x^4 - 5x^2 + 4$.

Try factoring:

$$f(x) = (x^2 - 4)(x^2 - 1) = (x - 2)(x + 2)(x - 1)(x + 1)$$

Thus $f(x)$ is reducible over $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, and $\mathbb{C}$.

Example 4: Irreducibles in $\mathbb{F}_2[x]$ Let $f(x) = x^2 + x + 1$ in $\mathbb{F}_2[x]$.

- $\mathbb{F}_2 = \{0, 1\}$.
- Evaluate at $x = 0$: $f(0) = 1$, $x = 1$: $f(1) = 1 + 1 + 1 = 1$.
- No root in $\mathbb{F}_2$, so $f(x)$ is irreducible over $\mathbb{F}_2$.

This is crucial in coding theory and finite field extensions.

Example 5: Gauss's Lemma Let $f(x) = 2x^2 + 3x + 1 \in \mathbb{Z}[x]$.

- It is reducible over $\mathbb{Q}$: $f(x) = (2x + 1)(x + 1)$.
- Gauss's Lemma: If $f \in \mathbb{Z}[x]$ is reducible in $\mathbb{Q}[x]$, then it is reducible in $\mathbb{Z}[x]$.

This example shows how rational factorization implies integer factorization in UFDs.

Conclusion: Factorization in polynomial rings extends the familiar concept of prime decomposition. Understanding irreducibility, applying tools like Eisenstein's criterion or Gauss's Lemma, and exploring factorizations over different base rings is essential in algebraic number theory, Galois theory, and beyond.

Chapter 4

Symbolic Computation

The content in this section is based on the foundational text ***Computer Algebra and Symbolic Computation: Elementary Algorithms*** by **Joel S. Cohen** [1]. Over the next five subsections, we introduce and explore the theory, concepts, and algorithms central to symbolic computation. The topics include an overview of computer algebra systems (CAS), core symbolic concepts such as expression trees and simplification rules, the recursive structure of symbolic expressions, symbolic algorithms for algebraic manipulation, and a practical understanding of recursion in symbolic systems.

Throughout this section, we have made a consistent effort to connect symbolic computation theory to the theme of this dissertation: analyzing and implementing digit-based transformations, specifically flipping numbers. Each subsection contains examples and explanations that directly relate symbolic methods to this problem domain. These connections demonstrate how symbolic tools can be used to represent, manipulate, and reason about flipped numeric forms, as well as to detect palindromes and algebraic equivalence through symbolic rewriting.

To better illustrate and reinforce the theoretical content, practical examples (often using SymPy or simplified pseudocode) have been incorporated. These examples highlight real use cases and offer an applied understanding of abstract symbolic principles.

One important reason for selecting symbolic recursion as a core theme is that recursion also forms the heart of the algorithm used in my C++ implementation for flipping number sequences. The recursive logic used in symbolic systems parallels the recursive design of our custom palindrome-checking and number-flipping functions, making the study of symbolic recursion both relevant and insightful for bridging theory with implementation.

4.1 Introduction to Symbolic Computation and Computer Algebra Systems

Definition: Symbolic computation, also referred to as computer algebra, is the area of computer science and mathematics concerned with the development of algorithms and software for manipulating mathematical expressions and objects in symbolic form, rather than as numerical approximations.

Unlike numerical computation, which produces approximated results using floating-point arithmetic, symbolic computation deals with exact representations. For instance, it retains expressions like $\sqrt{2}$ or $\frac{1}{3}$ in their exact form instead of approximating them as 1.414 or 0.333. Similarly, trigonometric expressions such as $\sin(\pi)$ are kept unevaluated or reduced to exact values like 0, instead of returning a floating-point approximation.

Motivation and Importance: Symbolic computation plays a foundational role in enabling computers to perform exact algebraic manipulations. Tasks include simplifying expressions, solving equations, computing integrals and derivatives symbolically, and manipulating polynomials and rational functions. This is particularly useful in contexts where precision is critical or where mathematical structure must be preserved. It also provides a basis for manipulating symbolic equations in areas like control systems, signal processing, algebraic geometry, and cryptography.

One of the key motivations is to replicate how mathematicians reason about formulas, identities, and relationships. Symbolic systems allow users to not just evaluate expressions but to transform, simplify, and restructure them in useful ways.

Computer Algebra Systems (CAS): Computer Algebra Systems (CAS) are software systems designed to carry out symbolic mathematics. Some well-known CAS include:

- **Maple -** Widely used in education and research for algebra, calculus, and equation solving.
- **Mathematica -** Known for symbolic manipulation, visualizations, and applied mathematics.
- **SymPy -** A Python library for symbolic computation, open-source and highly extensible.
- **Maxima** and **SageMath -** Free tools used in both teaching and research.

These systems provide interfaces for inputting symbolic expressions and manipulating them using built-in rules, transformation functions, and algorithms. They are not only essential in pure mathematics but also play an increasing role in fields like machine learning, artificial intelligence, robotics, and software verification.

Applications in Algebra and Computation: Symbolic computation supports many fields of mathematics and engineering:

- **Solving polynomial equations** symbolically.
- **Computing exact derivatives and integrals** in calculus.
- **Simplifying algebraic identities** and expressions.
- **Symbolic linear algebra**: computing eigenvalues, determinants, and matrix inverses symbolically.
- **Generating code for numerical algorithms** from symbolic expressions.

Symbolic techniques are also widely used to reason about recursive definitions and loops in programming languages, enabling optimizations and symbolic execution in compilers.

Connection to Dissertation: Modeling Flipping Operations Symbolically: Symbolic computation has particular relevance to this dissertation's theme of flipping numbers. Treating a number or digit sequence as a symbolic object allows one to:

- Represent a number as a symbolic expression tree (e.g., digits as leaves, positional powers as structure).
- Apply symbolic rewrite rules to model a "flip" operation.
- Simplify, compare, or analyze the flipped number algebraically.

For example, flipping a number like 123 symbolically can be treated as transforming $1 \cdot 10^2 + 2 \cdot 10^1 + 3$ into $3 \cdot 10^2 + 2 \cdot 10^1 + 1$. CAS tools can then be used to explore properties of such transformations (e.g., palindromes, reversibility, invariants under flipping).

Example 1: Symbolic Expression Transformation in SymPy In Python's SymPy:

```
from sympy import symbols, expand
x = symbols('x')
expr = (x + 1)**3
expand(expr)
```

This outputs:

```
x**3 + 3*x**2 + 3*x + 1
```

Rather than approximating $(x+1)^3$, the symbolic system expands it using exact algebraic rules. The same principles can be applied to number-based expressions, digit manipulation, or recursive flip operations.

Example 2: Digit Flipping as Symbolic Transformation Suppose we define a function `flip(n)` that reverses the digits of a number n. For $n = 321$, we can represent this as:

$$321 = 3 \cdot 10^2 + 2 \cdot 10^1 + 1, \quad \text{flip}(321) = 1 \cdot 10^2 + 2 \cdot 10^1 + 3$$

Using a symbolic expression system, we can write a general transformation rule:

$$\text{flip}\left(\sum_{i=0}^{k} d_i \cdot 10^i\right) = \sum_{i=0}^{k} d_i \cdot 10^{k-i}$$

This transformation preserves information and allows exploration of flipped-number properties algebraically. For example, checking whether a number is a palindrome symbolically becomes checking whether the expression equals its flipped counterpart.

Conclusion: Symbolic computation provides a framework for exact, rule-based manipulation of mathematical expressions. It forms the computational foundation of many modern mathematical systems and is highly applicable to structural tasks like modeling number flips and expression transformations. It enables the creation and analysis of symbolic systems that retain structure, reveal properties, and support automated reasoning. In the next section, we will explore the core concepts that make symbolic manipulation possible.

4.2 Core Concepts of Symbolic Computation

Symbolic computation relies on several foundational concepts that differentiate it from numerical methods. These include symbolic representations, substitution models, simplification rules, and expression manipulation using data structures. Understanding these core components is crucial to leveraging symbolic systems effectively for tasks such as expression rewriting, equation solving, or number transformation.

Symbolic vs. Numeric Representation: At its core, symbolic computation involves representing mathematical entities in symbolic form rather than evaluating them immediately. For example, the expression $x^2 + 2x + 1$ is retained as-is in a symbolic system until simplification or transformation is explicitly requested. In contrast, numerical systems evaluate expressions with concrete values (e.g., $x = 3$), often resulting in approximate results.

Symbolic expressions are not just values; they are structured objects that can be manipulated according to algebraic rules. This ability allows symbolic computation to support abstract mathematical reasoning, code generation, and structural pattern matching — features not available in pure numerical systems.

Expression Trees and Structure: In symbolic systems, expressions are typically represented as trees. Each operation (e.g., addition, multiplication) forms a node in the tree, and its operands form the subtrees. This tree-based structure allows recursive traversal and manipulation of subexpressions.
For instance, the expression $x^2 + 2x + 1$ can be visualized as:

$$\texttt{Add}\,(\texttt{Pow}(x, 2), \texttt{Mul}(2, x), 1)$$

Each component is an object with a specific kind (operator) and operands (subtrees). Symbolic systems provide built-in methods for accessing, modifying, or replacing parts of these trees. This makes operations like simplification or pattern rewriting straightforward.

Operators and Operands: Symbolic computation uses a consistent internal language to analyze expressions. For example, Maple and SymPy use functions like ‘type‘, ‘op‘, and ‘args‘ to retrieve structural information. Some standard symbolic operators include:

- **Kind** – Identifies the top-level operation (e.g., addition, multiplication).
- **Operands** – Returns the arguments or subcomponents of an expression.
- **Construct** – Combines a kind and operands to construct a new expression.

These functions allow symbolic software to recursively analyze expressions, break them into parts, and apply rules selectively — much like manipulating syntax trees in programming languages.

Substitution and Rewriting: Substitution is a fundamental tool in symbolic manipulation. It involves replacing one symbolic expression with another according to a

predefined rule. For example:

$$x^2 + 2x + 1 \Rightarrow (x + 1)^2$$

Symbolic systems can apply rules like:

$$\texttt{substitute}(x^2 + 2x + 1, x^2 + 2x + 1 \mapsto (x + 1)^2)$$

More generally, rewriting can be controlled using pattern matching, enabling the user to specify transformation templates. This is particularly powerful in exploring symmetry or modeling algebraic transformations.

Simplification Strategies: Simplification is the process of reducing an expression to a simpler or more canonical form. Symbolic systems offer different simplification modes, such as:

- **Algebraic simplification**: Combining like terms, factoring, expanding.
- **Trigonometric simplification**: Using identities like $\sin^2 x + \cos^2 x = 1$.
- **Rational simplification**: Canceling common terms in rational expressions.

Simplification is context-dependent and symbolic systems often include heuristics to choose the most meaningful form. For instance, some systems will preserve factored form while others may return an expanded version, depending on the function invoked (e.g., ‘expand‘, ‘factor‘, ‘simplify‘).

Application to Digit-Flipping and Transformation: The structure and control provided by symbolic computation directly support modeling of digit-based transformations such as flipping. For example, suppose we want to flip the number 231. We represent it symbolically as:

$$2 \cdot 10^2 + 3 \cdot 10^1 + 1 \cdot 10^0$$

By using symbolic rewrite rules and operand access, we can define a transformation that reverses the order of the terms:

$$\sum_{i=0}^{k} d_i \cdot 10^i \mapsto \sum_{i=0}^{k} d_i \cdot 10^{k-i}$$

Symbolic substitution and expression traversal make this rule implementable in systems like SymPy or Maple. Moreover, pattern-matching rules can ensure that the transformation only applies when the structure matches a digit-power representation.

Example: Symbolic Substitution with SymPy In Python's SymPy:

```
from sympy import symbols, expand, simplify
x = symbols('x')
expr = x**2 + 2*x + 1
simplified = simplify(expr)
```

The system returns:

```
(x + 1)**2
```

Now consider digit flipping:

```
from sympy import Symbol
n = [2, 3, 1]  # digits of 231
expr = sum(d * 10**i for i, d in enumerate(reversed(n)))
flip_expr = sum(d * 10**i for i, d in enumerate(n))
```

Here, ‘expr‘ is the symbolic form of 132 (flipped 231). The use of symbolic powers and sums means we can now perform further algebraic reasoning — for instance, comparing expressions, checking equality, or analyzing palindromes.

Conclusion: Core concepts such as expression trees, substitution, simplification, and structural traversal form the backbone of symbolic computation. They enable not just symbolic algebra but also computational transformations like digit flipping. By treating

expressions as structured, manipulable objects, symbolic systems empower flexible, programmable manipulation of mathematical ideas — exactly the kind of system needed to model the symmetry and reversibility themes of this dissertation.

4.3 Recursive Structure of Symbolic Expressions

One of the most powerful features of symbolic computation is its recursive representation of expressions. In this model, every expression is built from smaller subexpressions, forming a hierarchical tree structure. This design allows for elegant, efficient manipulation of mathematical objects, since transformations and analysis can be applied recursively to components of an expression.

Recursive Definition of Expressions: Symbolic expressions are inherently recursive. A number is an expression, a variable is an expression, and any operator applied to one or more expressions forms a larger expression. This forms the basis of symbolic trees, where:

- Leaf nodes represent constants or variables.
- Internal nodes represent operations (e.g., addition, multiplication, exponentiation).

For instance, the expression $3(x^2 + 2x + 1)$ can be visualized as a recursive structure:

$$\texttt{Mul}(3, \texttt{Add}(\texttt{Pow}(x, 2), \texttt{Mul}(2, x), 1))$$

Each node can be decomposed into its parts recursively. This makes symbolic computation naturally compatible with recursive algorithms such as simplification, differentiation, or substitution.

Recursive Traversal of Expressions: Symbolic systems traverse expressions recursively to analyze or transform them. A traversal might:

- Identify a subexpression that matches a pattern.
- Apply a simplification or transformation rule.

- Rebuild the expression from its transformed parts.

This traversal is often implemented using a post-order strategy: children are processed first, then the parent node. For example, to simplify $3(x+1)^2$, the system would:

1. Visit $(x+1)$,
2. Compute $(x+1)^2$,
3. Multiply the result by 3.

Such recursion enables modularity and clarity in symbolic manipulation.

Manipulating Trees in Symbolic Systems: Most symbolic systems provide tools to interact with expression trees. In Maple or SymPy, common functions include:

- `kind(expr)` – returns the top-level operation.
- `args(expr)` – returns a list of operands.
- `preorder` or `postorder` – traversal functions.
- `replace` or `subs` – applies recursive pattern replacement.

These tools allow developers to write rules that recursively simplify expressions, perform transformations, or gather specific subterms (e.g., all x^2 terms in a polynomial).

Application to Number Flipping and Expression Reversal: The recursive structure is especially useful for digit manipulation. For example, the number 231 can be represented symbolically as:

$$2 \cdot 10^2 + 3 \cdot 10^1 + 1 \cdot 10^0$$

This expression is already a tree: each term is a multiplication node, and the entire number is an addition node. To flip this number, we traverse the expression and rearrange the powers of 10:

$$\sum_{i=0}^{k} d_i \cdot 10^i \mapsto \sum_{i=0}^{k} d_i \cdot 10^{k-i}$$

This can be implemented as a recursive function:

```
def flip_expr_tree(terms):
    n = len(terms)
    return [d * 10**(n-1-i) for i, d in enumerate(terms)]
```

The symbolic representation allows such a function to work on abstract digits, not just evaluated values. This flexibility is key for algebraic reasoning and pattern-based flipping.

Example: Recursively Expanding and Simplifying Expressions: Consider this Python example using SymPy:

```
from sympy import symbols, expand, simplify
x = symbols('x')
expr = 3 * (x + 1)**2
expanded = expand(expr)     # 3*x**2 + 6*x + 3
simplified = simplify(expr) # Returns original if already simplified
```

The expansion uses recursion internally: it first expands $(x+1)^2$ and then distributes the multiplication. These recursive passes make the system robust for arbitrary complexity.

Practical Benefits of Recursive Structure: The recursive representation has several practical advantages:

- **Modularity:** Functions can operate on parts of expressions without full re-evaluation.
- **Efficiency:** Recursive algorithms can stop early, cache results, or apply optimizations locally.
- **Expressiveness:** Complex structures like nested radicals, continued fractions, or sums can be handled cleanly.

In contexts like this dissertation, these benefits are critical for analyzing the effects of flipping transformations on symbolic expressions and evaluating whether properties (like symmetry or palindromicity) are preserved.

Dealing with Nested Transformations: Recursive systems also support nested transformations, which occur when one rewrite triggers another. For example:

$$\texttt{Expand}(3(x+1)^2) \Rightarrow 3x^2 + 6x + 3 \Rightarrow \text{Group terms} \Rightarrow \text{Factor}$$

These chains can be modeled using recursive control structures, enabling symbolic engines to evaluate and restructure deeply nested constructs without manual intervention.

Conclusion: Symbolic expressions are naturally recursive, making them ideal for structural transformations like those involved in digit flipping or equation manipulation. Expression trees provide a robust foundation for traversal, substitution, and simplification. By leveraging recursive functions and pattern-based rules, symbolic systems can manipulate even complex algebraic objects cleanly and efficiently—aligning well with the computational goals of this dissertation.

4.4 Symbolic Algorithms and Mathematical Computation

Symbolic computation is not merely about representing expressions; it is about transforming and solving them using systematic procedures. These procedures, known as symbolic algorithms, operate on algebraic structures to perform tasks such as simplification, differentiation, integration, equation solving, and polynomial manipulation. Unlike numerical algorithms that yield approximate results, symbolic algorithms maintain exactness and structural clarity.

What Are Symbolic Algorithms? Symbolic algorithms are designed to manipulate mathematical objects in their symbolic form. They follow deterministic rules that preserve the identity and properties of expressions while transforming them into more desirable or usable forms. A symbolic algorithm must consider the algebraic rules governing the expressions involved — such as commutativity, associativity, distributivity, and identity

laws — to carry out valid transformations.

Common symbolic algorithms include:

- Polynomial expansion and factorization
- Simplification using known identities
- Symbolic differentiation and integration
- Solving equations and inequalities symbolically
- Computation of greatest common divisors (GCDs) of polynomials

These algorithms often work recursively on expression trees and make use of pattern matching and term rewriting systems.

Polynomial Algorithms: Expansion and Factorization: Polynomials are among the most studied symbolic objects. A basic algorithm expands a factored polynomial into its standard form. For instance:

$$(x+1)^3 \Rightarrow x^3 + 3x^2 + 3x + 1$$

The reverse process — factorization — reduces a polynomial to a product of irreducible components. For example:

$$x^2 - 5x + 6 \Rightarrow (x-2)(x-3)$$

These operations are useful for solving algebraic equations, where factoring a polynomial helps identify its roots.

Symbolic systems such as Maple, Mathematica, and SymPy implement efficient algorithms for expansion and factorization. These algorithms are not only algebraically rigorous but also optimized for performance.

Symbolic Differentiation and Integration: Symbolic differentiation is one of the simplest yet most powerful features of symbolic computation. Given an expression $f(x)$,

the algorithm applies known derivative rules (product rule, chain rule, etc.) recursively on subexpressions.

Example:

$$\frac{d}{dx}(x^3 + 2x^2 + x + 1) = 3x^2 + 4x + 1$$

Symbolic integration is more complex, as it may involve special functions and pattern matching. Algorithms like Risch's algorithm attempt to determine whether a symbolic integral exists in closed form.

These operations are critical for solving calculus-based problems, especially in contexts like control theory, physics, and engineering.

Equation Solving and Simplification: Another class of symbolic algorithms focuses on solving equations. For example, given:

$$x^2 - 4 = 0$$

a symbolic system can return the exact solutions:

$$x = 2, -2$$

This is in contrast to numerical solvers which might yield $\pm 1.999...$ due to floating-point limitations. Symbolic equation solvers may also return parametric solutions for underdetermined systems, or families of solutions when the input is symbolic.

Simplification is a broad algorithmic domain. It includes combining like terms, removing zero terms, rationalizing denominators, and reducing expressions to canonical form. Simplification plays a key role in comparing expressions, optimizing code generation, and identifying symmetries.

GCD Computation of Polynomials: In symbolic systems, polynomials over a ring like $\mathbb{Z}[x]$ are treated similarly to integers. Just as the GCD of two integers reveals shared

factors, the GCD of two polynomials provides the common divisors. For instance:

$$\gcd(x^3 - 3x^2 + 2x, x^2 - 3x + 2) = x - 1$$

This result helps in simplifying rational expressions, solving polynomial equations, and determining algebraic dependencies.

GCD algorithms are often based on Euclid's algorithm adapted for polynomial rings. These are recursive algorithms that perform repeated division with remainder until the remainder is zero.

Application to Digit-Based Transformations: In this dissertation's context of flipping numbers, symbolic algorithms can model such operations explicitly. Consider a number represented as:

$$N = d_2 \cdot 10^2 + d_1 \cdot 10^1 + d_0$$

We can use symbolic rewriting to produce its flipped form:

$$\text{flip}(N) = d_0 \cdot 10^2 + d_1 \cdot 10^1 + d_2$$

Now suppose we want to simplify an expression involving both N and $\text{flip}(N)$, or determine whether $\text{flip}(N) = N$ (i.e., check if N is a palindrome). Symbolic simplification and substitution algorithms can handle such manipulations, using pattern-based rewriting.

Moreover, we can define functions like:

```
def is_palindrome(n):
    digits = [int(d) for d in str(n)]
    expr = sum(d * 10**i for i, d in enumerate(reversed(digits)))
    return n == expr
```

Although this function is numeric, symbolic systems can generalize it for variables (e.g., a, b, c for digits) and perform algebraic checks, like whether $100a + 10b + c = 100c + 10b + a$.

Conclusion: Symbolic algorithms are the computational backbone of computer algebra systems. They offer deterministic, exact methods to manipulate mathematical objects, solve equations, and reveal structure. For this dissertation, symbolic algorithms allow the formalization and analysis of digit flipping operations, symmetry detection, and algebraic equivalence. Their robustness and exactness make them ideal tools for symbolic reasoning in number transformations.

4.5 A Practical View of Recursion in Symbolic Computation

Recursion is not only a theoretical concept in symbolic computation — it is a practical tool that underpins how expressions are represented, analyzed, and transformed. Most symbolic computation systems use recursion as the default mechanism to traverse expression trees, apply transformations, and evaluate results. In this subsection, we explore how recursion is implemented, what makes it efficient, and how it helps in symbolic tasks like expression simplification, transformation, and digit manipulation.

What is Recursion in Practice? Recursion refers to the process where a function calls itself to solve smaller instances of a problem until a base case is reached. In symbolic computation, this structure aligns naturally with expression trees. Since symbolic expressions are recursively defined, recursive functions can efficiently operate on their structure.
Consider an expression like:

$$3(x+1)^2 = 3 \cdot (x^2 + 2x + 1)$$

To expand this, the system first expands $(x+1)^2$, then multiplies each term by 3. Each step is a recursive application of transformation rules: expand the subexpression, apply distributivity, and reconstruct the parent expression.

Stack Frames and Depth in Symbolic Systems: Each recursive call in symbolic computation corresponds to a new stack frame. Efficient systems manage recursion depth carefully to avoid overflow or inefficiency. Some symbolic systems optimize recursion using tail recursion, memoization, or iterative equivalents in cases where large expressions are processed.

For example, evaluating a deeply nested expression such as:

$$f(x) = (((x + 1) + 1) + 1) + ... + 1$$

involves multiple recursive calls. Symbolic systems like Maple or Mathematica are optimized to recognize patterns and collapse such expressions using internal simplification routines before triggering unnecessary recursion.

Recursive Simplification and Rule Application: Most simplification routines are inherently recursive. Given an expression, a symbolic engine will:

1. Simplify each subexpression.
2. Apply simplification rules at the current node.
3. Rebuild the expression with simplified subparts.

This process continues recursively until the entire expression is simplified.

Example: Simplify $3(x + 1)^2$ using SymPy:

```
from sympy import symbols, expand
x = symbols('x')
expr = 3 * (x + 1)**2
result = expand(expr)
```

Internally, this calls the expand function recursively on $(x+1)^2$, then distributes 3 across the result. Each call works on a smaller part of the tree until a base case (e.g., constants or variables) is reached.

Recursive Pattern Matching: Symbolic systems often use recursion to apply pattern matching rules. A rule like:

$$a \cdot 0 \Rightarrow 0$$

can be applied recursively to subparts of an expression. For example, in:

$$(a + b)(c \cdot 0)$$

the engine will:

- Recursively visit $c \cdot 0$ and reduce it to 0.
- Multiply $a + b$ by 0.
- Return 0 as the final result.

Such recursive traversal ensures that rules are applied consistently and comprehensively.

Recursive Digit Transformation: Connection to Flipping Numbers: Recursion is especially useful when performing operations like flipping digits. Consider the number 321. A recursive approach to flipping could process the last digit, then flip the rest:

```
def flip_digits(n):
    if n < 10:
        return str(n)
    return str(n % 10) + flip_digits(n // 10)
```

This function builds the flipped number from right to left. Symbolic equivalents can be written where the digits are variables (e.g., a, b, c), and the output is an expression:

$$\text{flip}(a \cdot 10^2 + b \cdot 10 + c) = c \cdot 10^2 + b \cdot 10 + a$$

This form allows symbolic systems to reason about flips algebraically and apply them to general forms.

Recursive Algorithms in Expression Evaluation: Another key use of recursion is in symbolic evaluation. Consider computing the derivative of a nested function:

$$\frac{d}{dx}(x^2 + \sin(x^3))$$

The system recursively applies the chain rule to $\sin(x^3)$, first differentiating the outer sin, then multiplying by the derivative of x^3. The final result is:

$$2x + 3x^2 \cdot \cos(x^3)$$

This is constructed recursively by applying differentiation rules to inner and outer functions.

Benefits of Recursive Implementation: Recursion in symbolic computation offers the following advantages:

- **Simplicity**: Recursive functions closely mirror the mathematical structure of expressions.
- **Clarity**: They are easier to understand and reason about than iterative equivalents in many cases.
- **Modularity**: Functions can process each part independently, making them reusable.
- **Powerful abstraction**: Recursion enables generalized rules to be applied to arbitrarily complex expressions.

Symbolic systems often combine recursion with rule-based transformation engines to implement intelligent simplification, normalization, and structure analysis.

Conclusion: Recursion is a practical engine that powers symbolic computation. It enables symbolic systems to traverse, analyze, and transform expression trees efficiently. From simplification and evaluation to pattern matching and digit transformations, recursion underlies the flexibility and power of computer algebra systems. In the context of

this dissertation, recursion supports the symbolic modeling of number flipping operations and their algebraic analysis, making it an indispensable computational technique.

Chapter 5

Ideal Membership

In this chapter, we will focus on the concept of ideal membership, which serves as a crucial bridge between the abstract theory of commutative algebra and the practical computations required in symbolic algebra systems. The ideal membership problem asks whether a given polynomial belongs to an ideal generated by other polynomials, a question that lies at the heart of computational algebraic geometry. Its significance extends beyond pure algebra, as it provides the formal link between the mathematical framework of invariant rings and quotient constructions introduced in earlier sections, and their application to geometric flips. From a computational standpoint, ideal membership is fundamental because algorithms such as Gröbner basis methods (developed from Buchberger's algorithm) provide concrete procedures for resolving these problems. Thus, it is precisely through the lens of ideal membership that symbolic computation connects with the birational transformations mentioned in chapter **6**.

The material in this chapter is based on the standard reference ***Ideals, Varieties, and Algorithms*** by **David Cox, John Little, and Donal O'Shea** [3], which provides both the theoretical foundation and algorithmic techniques for tackling ideal membership problems. This text not only develops the algebraic background but also introduces Gröbner basis algorithms, which will later be used to illustrate the computational aspect of flips in practice.

5.1 Introduction to Ideal Membership

The concept of *ideal membership* lies at the heart of computational algebraic geometry and serves as a fundamental bridge between abstract algebra and symbolic computation. At its core, the ideal membership problem asks the following: given a set of polynomials $g_1, g_2, \ldots, g_m \in k[x_1, \ldots, x_n]$ that generate an ideal $I = \langle g_1, \ldots, g_m \rangle$, and another polynomial $f \in k[x_1, \ldots, x_n]$, can we decide whether f belongs to I? In other words, can we determine if there exist polynomials $h_1, \ldots, h_m \in k[x_1, \ldots, x_n]$ such that

$$f = h_1 g_1 + h_2 g_2 + \cdots + h_m g_m?$$

This deceptively simple question encapsulates the interaction between algebraic structures and computational procedures. On one side, the problem belongs to the realm of commutative algebra, where ideals and their properties are studied abstractly. On the other, it directly motivates the development of algorithms in symbolic computation, which allow us to manipulate polynomial systems and answer membership queries systematically.

Why Ideal Membership Matters

Ideal membership is not merely an algebraic curiosity; it is the computational engine underlying many tasks in algebraic geometry and its applications. For example, when solving systems of polynomial equations, determining whether a candidate polynomial identity holds within the solution set amounts to checking membership in a corresponding ideal. Likewise, elimination theory, primary decomposition, and the computation of varieties are all reducible to questions of membership.

From the perspective of symbolic computation, ideal membership plays a role analogous to divisibility in the integers: just as algorithms such as the Euclidean algorithm allow us to decide whether one integer divides another, Gröbner basis methods provide a systematic way to decide ideal membership for polynomials in several variables. This analogy highlights the algorithmic nature of the subject: ideals allow us to organise polynomial equations, and membership tests allow us to manipulate them in a controlled way.

The Computational Challenge

The membership problem becomes highly nontrivial when dealing with several variables. In one variable, the Euclidean algorithm provides a straightforward method to determine divisibility. However, in the multivariate case, polynomial division is not uniquely defined without an ordering on the monomials, and naive strategies fail to provide consistent answers. This difficulty gave rise to the theory of Gröbner bases, introduced by Buchberger in the 1960s, which not only provide a canonical generating set for an ideal but also transform the membership question into a tractable algorithmic problem. The design of these algorithms marked a turning point in algebraic geometry, making it accessible to computation and broadening its applications to engineering, robotics, coding theory, and computer science.

Applications to Geometric Flips

The study of geometric flips in algebraic geometry requires us to navigate between algebraic invariants and geometric models. For example, when analysing the coordinate ring associated with a graded vector such as $(1, 1, -1, -1)$, the classification of generators and relations depends on membership in specific ideals. Determining whether one relation follows from others, or whether certain binomial relations vanish on a variety, is precisely an ideal membership problem. Thus, when we later analyse the flip diagram

$$X^- \longrightarrow X \longleftarrow X^+,$$

the role of ideals and their membership properties becomes indispensable in verifying whether the corresponding coordinate rings and morphisms satisfy the defining conditions of a flip. In this sense, ideal membership provides the computational backbone for the transition from algebraic data to geometric insight.

5.2 Algorithms of Ideal Membership

The problem of *ideal membership* is central to computational algebraic geometry: given an ideal

$$I = \langle g_1, g_2, \ldots, g_m \rangle \subseteq k[x_1, \ldots, x_n]$$

and a polynomial $f \in k[x_1, \ldots, x_n]$, decide whether $f \in I$. Equivalently, can we determine whether there exist polynomials $h_1, \ldots, h_m$ such that

$$f = h_1 g_1 + h_2 g_2 + \cdots + h_m g_m?$$

At first glance this problem resembles the familiar case of divisibility in the integers or univariate polynomials. However, the complexity increases dramatically in the multivariate case. While the Euclidean algorithm provides an efficient way to check divisibility in one variable, no such straightforward analogue exists for several variables. This is where algorithms of symbolic computation become indispensable.

5.2.1 Monomial Orderings

A key obstacle in the multivariate case is that polynomial division is no longer uniquely defined. To resolve this ambiguity, we introduce monomial orderings, which impose a total order on the monomials of $k[x_1, \ldots, x_n]$. Two of the most commonly used orderings are:

- **Lexicographic order (lex):** Monomials are ordered by comparing exponents of x_1, then x_2, and so forth. For example, in $\mathbb{Q}[x, y]$, $x^2y > xy^5$ because $2 > 1$ in the exponent of x.
- **Graded lexicographic order (grlex):** Monomials are first ordered by total degree; ties are broken using lex order. For instance, $x^3y < x^2y^2$ since $\deg(x^3y) = 4 = \deg(x^2y^2)$ but x^3y is smaller in lex order.

These orderings provide the foundation for algorithms such as the multivariate division algorithm and Buchberger's algorithm. They allow us to define a unique leading term

$\mathrm{LT}(f)$ for each polynomial f, which is the "largest" monomial with respect to the chosen ordering. The notion of leading term is central to Gröbner bases, which solve the ideal membership problem algorithmically.

5.2.2 Division Algorithm and Gröbner Bases

Multivariate Division Algorithm

In one variable, the Euclidean division algorithm allows us to divide any polynomial $f(x)$ by another $g(x)$, yielding a quotient and remainder uniquely determined by degree. However, in several variables, polynomial division is more subtle. The main difficulty lies in the fact that there is no natural notion of degree order without additional structure. This motivates the use of monomial orderings, as introduced above, to provide a systematic way to identify leading terms.

Let $F = \{g_1, g_2, \ldots, g_m\}$ be a finite set of polynomials in $k[x_1, \ldots, x_n]$ with a fixed monomial order. The *multivariate division algorithm* takes as input a polynomial f and attempts to reduce it by the leading terms $\mathrm{LT}(g_i)$ of the divisors. At each step, if $\mathrm{LT}(g_i)$ divides the current leading term of the dividend, the algorithm subtracts a suitable multiple of g_i to cancel that term. This process continues until no further reduction is possible. The output of the algorithm is an expression

$$f = a_1 g_1 + a_2 g_2 + \cdots + a_m g_m + r,$$

where $a_i \in k[x_1, \ldots, x_n]$ and r is a remainder polynomial in which no monomial is divisible by any $\mathrm{LT}(g_i)$. Unlike the univariate case, the remainder r is not unique unless the generating set F has special properties. This lack of uniqueness is the source of ambiguity in ideal membership tests, since $f \in I$ if and only if a remainder of zero can be achieved. To overcome this, we require a more canonical choice of generators, leading to the notion of Gröbner bases.

Gröbner Bases

A Gröbner basis for an ideal $I \subseteq k[x_1, \ldots, x_n]$ with respect to a chosen monomial order is a generating set $G = \{g_1, \ldots, g_t\}$ such that the remainder of any polynomial f upon division by G is unique. Moreover, $f \in I$ if and only if this remainder is zero. This property makes Gröbner bases a powerful computational tool: they transform the ideal membership problem into a well-defined algorithmic procedure.

Formally, G is a Gröbner basis if for every $f \in I$, the leading term $\mathrm{LT}(f)$ is divisible by the leading term of some $g_i \in G$. Intuitively, this means that the Gröbner basis "controls" all leading terms in the ideal. As a result, the division algorithm relative to G produces a canonical remainder that does not depend on arbitrary choices made during reduction.

Example

Consider the ideal

$$I = \langle f_1, f_2 \rangle \subseteq \mathbb{Q}[x, y],$$

where $f_1 = x^2 - y$ and $f_2 = xy - 1$. Using lex order with $x > y$, the Gröbner basis for I can be computed as

$$G = \{\, xy - 1,\ y^2 - 1 \,\}.$$

Now suppose we wish to test whether $f = x^2y - y$ belongs to I. Dividing f by G under lex order yields remainder 0, so indeed $f \in I$. In contrast, if $r \neq 0$, we could conclude that $f \notin I$. This simple example illustrates how Gröbner bases turn ideal membership into a transparent computational procedure.

Significance

The existence of Gröbner bases ensures that the ideal membership problem can be decided algorithmically. This result, first proved by Buchberger in 1965, represents one of the most significant breakthroughs in computational algebra. Gröbner bases not only solve membership questions but also provide the foundation for elimination theory, solving systems of polynomial equations, and connecting algebra to geometry in the study of

varieties and birational transformations.

5.2.3 Buchberger's Algorithm and Applications

Buchberger's Algorithm

While Gröbner bases provide a canonical solution to the ideal membership problem, the key question is how to compute such a basis for a given ideal. This problem was solved by Buchberger in 1965 through the development of what is now called Buchberger's algorithm. The algorithm begins with an arbitrary generating set $F = \{f_1, f_2, \ldots, f_m\}$ of an ideal $I \subseteq k[x_1, \ldots, x_n]$ and systematically modifies it until it becomes a Gröbner basis. The essential idea of Buchberger's algorithm is to resolve ambiguities that arise in multivariate division. These ambiguities are captured by the construction of S-polynomials. Given two polynomials f_i and f_j, their S-polynomial is defined as

$$S(f_i, f_j) = \frac{\text{lcm}(\text{LT}(f_i), \text{LT}(f_j))}{\text{LT}(f_i)} \cdot f_i - \frac{\text{lcm}(\text{LT}(f_i), \text{LT}(f_j))}{\text{LT}(f_j)} \cdot f_j,$$

where lcm denotes the least common multiple of monomials. The purpose of the S-polynomial is to cancel leading terms of f_i and f_j, thereby revealing possible hidden relations among the generators.

Buchberger's algorithm proceeds by repeatedly computing S-polynomials of pairs of generators and reducing them with respect to the current set. If the reduction is nonzero, it is added to the set of generators. This process continues until all S-polynomials reduce to zero, at which point the set of generators forms a Gröbner basis for I.

Correctness and Termination

The correctness of Buchberger's algorithm follows from Buchberger's criterion: a set G of generators is a Gröbner basis if and only if the remainder of every S-polynomial $S(f_i, f_j)$ upon division by G is zero. Termination is guaranteed because each step of the algorithm strictly decreases the leading term ideal $\langle \text{LT}(G) \rangle$ with respect to the monomial order, and by Dickson's lemma, no infinite descending chain of monomial ideals exists in $k[x_1, \ldots, x_n]$.

Although the algorithm may be computationally expensive in practice, it provides a constructive proof that Gröbner bases always exist and can be computed. Subsequent refinements, such as the Buchberger criteria for avoiding unnecessary S-polynomials, as well as advanced variants like Faugère's F4 and F5 algorithms, have significantly improved efficiency in practical applications.

Applications of Gröbner Bases

The importance of Gröbner bases extends far beyond ideal membership. They provide a unifying algorithmic framework for a wide range of problems in algebra and geometry, including:

- **Solving polynomial systems:** A Gröbner basis can be used to triangularize a system of equations, analogous to Gaussian elimination for linear systems. This makes it possible to compute all solutions of an algebraic variety.

- **Elimination theory:** Gröbner bases allow variables to be eliminated systematically, providing an effective way to project varieties onto coordinate subspaces. This is particularly useful in applications such as robotics and kinematics, where geometric constraints must be simplified.

- **Geometric interpretation:** The canonical nature of Gröbner bases ensures that they encode the structure of varieties in a computationally accessible way. This makes them indispensable for studying birational maps and transformations such as flips.

- **Symbolic computation systems:** Modern computer algebra systems such as Macaulay2, Singular, Magma, and Maple implement Gröbner basis algorithms as a core functionality. These systems make it possible to automate complex ideal membership queries that would otherwise be infeasible by hand.

Connection to Geometric Flips

In the context of this dissertation, the significance of Buchberger's algorithm lies in its ability to connect symbolic computation to the study of geometric flips. For example, when analysing the invariant ring

$$R^0 = \mathbb{C}[xz, xt, yz, yt],$$

relations such as $us - vw = 0$ must be verified to determine the defining equations of the corresponding variety. Such verifications are nothing more than ideal membership problems. Gröbner basis computations provide a systematic way to confirm whether a proposed relation follows from a generating set, thereby establishing the algebraic consistency of the geometric construction. In this way, algorithms of ideal membership serve as the computational underpinning of the birational transformations that define flips.

Conclusion

To summarise, the algorithms of ideal membership transform abstract algebraic questions into practical computational procedures. The multivariate division algorithm highlights the role of monomial orderings, Gröbner bases resolve the ambiguity of remainders, and Buchberger's algorithm provides a concrete method for constructing such bases. Together, these tools make the ideal membership problem algorithmically solvable, forming the foundation of computational algebraic geometry. Moreover, they provide the crucial bridge between symbolic computation and the geometric study of flips, ensuring that the algebraic theory developed in earlier chapters can be effectively applied to the geometric problems at the heart of this dissertation.

5.3 The Role of Ideal Membership

One of the central bridges between the abstract algebraic theory discussed in this series and practical symbolic computation is the concept of **ideal membership**. In computational algebra, especially in computer algebra systems like Magma, Singular, or

MACAULAY2, determining whether a given polynomial belongs to an ideal generated by other polynomials is a fundamental task.

For example, when we construct quotient rings such as

$$\widehat{R} = \mathbb{C}[a, b, c, d, x, y]/(ad - bc, ay - bx, cy - dx),$$

we are essentially encoding a set of algebraic relations as an ideal $I = (ad-bc, ay-bx, cy-dx)$. The quotient ring $\widehat{R}$ consists of all polynomials in the variables a, b, c, d, x, y, modulo this ideal. From a computational point of view, any operation in $\widehat{R}$ involves checking whether two polynomials are equivalent modulo the generators of I, which amounts to testing if their difference lies in I, that is, solving the ideal membership problem.

This is where symbolic computation plays a key role. Algorithms such as Buchberger's algorithm for Gröbner bases are designed to answer ideal membership queries efficiently. When we are given a polynomial f, and an ideal I generated by $g_1, g_2, \ldots, g_n$, the question "Is $f \in I$?" becomes algorithmically solvable using these computational techniques.

Thus, the algebraic structures explored in these subsections: graded rings, invariant rings, quotient rings naturally lead into the realm of symbolic computation, where ideal membership forms the core of many practical implementations in computational algebraic geometry.

Ideal Membership and Symbolic Computation

In the broader context of symbolic computation, the ideal membership problem is one of the paradigmatic algorithmic challenges. Symbolic computation is concerned with performing exact algebraic manipulations on a computer, as opposed to numerical approximation. Tasks such as factorisation, simplification, and solving systems of equations all rely, at their deepest level, on membership tests in polynomial ideals. Thus, ideal membership is not just a technical detail: it represents the conceptual meeting point where algebraic theory is made effective by computation.

Furthermore, ideal membership provides the essential link between the algebraic founda-

tions laid in earlier chapters (group theory and ring theory) and the geometric ideas of flips. In particular, the study of invariant rings, quotient structures, and coordinate rings of varieties relies on knowing whether certain polynomials can be expressed in terms of generators of an ideal. This is where computational methods come into play: without an algorithm for membership, the algebraic theory would remain abstract and disconnected from its geometric applications.

Chapter 6

Geometric Flips

6.1 Graded Rings, Ideals, and Quotients

In this subsection, we delve into the algebraic structures that underpin the notion of a geometric flip, with a particular focus on how rings and ideals encode transformations that can be entirely realized using sets of integers. The content presented here builds on the idea that flips can be interpreted through changes in coordinate systems, described by graded polynomial rings and their associated quotient structures.

Graded Ring Construction: Let us consider the grading vector $(1, 1, -1, -1)$, which we assign to four variables x, y, z, t. This grading defines a $\mathbb{Z}$-grading on the polynomial ring

$$R = \mathbb{C}[x, y, z, t]$$

where each variable has the following degree:

$$\deg(x) = \deg(y) = 1, \quad \deg(z) = \deg(t) = -1$$

This grading partitions elements of R according to their total degree. In particular, the degree zero part, denoted R°, consists of all polynomials in R whose monomials have total degree zero under the above assignment.

A standard way to construct R° is to consider all monomials of degree zero formed by

combining positive and negative degree variables. In our example, this yields:

$$R^\circ = \mathbb{C}[xz, xt, yz, yt]$$

Let us denote:

$$u = xz, \quad v = xt, \quad w = yz, \quad s = yt$$

Then R° becomes:

$$R^\circ = \mathbb{C}[u, v, w, s]/(us - vw)$$

The relation $us - vw = 0$ is a defining relation among the generators and encodes a geometric constraint in the corresponding affine variety. This quotient ring defines a hypersurface in $\mathbb{A}^4$ and represents a singular algebraic variety associated with the flip.

Positive and Negative Degree Rings: To better understand the flip structure, we introduce two related subrings:

- The ring of non-negative degrees:

$$R^+ = R^\circ[x, y]$$

- The ring of non-positive degrees:

$$R^- = R^\circ[z, t]$$

This decomposition reflects how different local coordinate patches can be glued to form a complete birational transformation. Each patch corresponds to a different region of the space. One for the "incoming" configuration and one for the "flipped" configuration. This mirrors the idea in the flip paper where different GIT quotients (e.g., X_- and X_+) are constructed from open subsets and then related via a birational map.

Ideals and Their Role in Quotients: The board discussion then moved to ideals, which are subsets of a ring that absorb multiplication and enable quotient constructions—

essential in forming coordinate rings of algebraic varieties.

Given a ring R, an ideal $I \subseteq R$ satisfies:

1. $a, b \in I \Rightarrow a + b \in I$

2. $r \in R, a \in I \Rightarrow ra \in I$

We denote the ideal generated by an element $r \in R$ as:

$$(r) = \{fr \mid f \in R\}$$

Using this, we define the quotient ring R/I, where elements of R are grouped into equivalence classes based on the ideal. These classes are written using square brackets, such as $[\alpha], [\beta]$, with the equivalence relation:

$$[\alpha] = [\beta] \iff \alpha - \beta \in I$$

Arithmetic operations in the quotient ring are defined as:

$$[\alpha] + [\beta] = [\alpha + \beta], \quad [\alpha][\beta] = [\alpha \cdot \beta]$$

This structure forms the foundation for symbolic computation on algebraic varieties, particularly when investigating how singularities change under flips.

Relevance to Geometric Flips: The entire construction showcases how flips can be modeled purely algebraically using a change of grading and ideal-based equivalence classes. The flip transforms one coordinate patch to another via the relation $us = vw$, which is intrinsic to the ring structure. As highlighted in Corti's paper **"What is a Flip?"**, such transformations called codimension-2 surgeries are central in the minimal model program and can often be expressed through changes in graded coordinate rings and their ideals.

6.2 Algebraic and Geometric Perspective of Flips

Let

$$R = \mathbb{R}[x, y]/\langle y - x^2 \rangle$$

This is the coordinate ring of the parabola $y = x^2$ in $\mathbb{R}^2$. It consists of polynomial functions on the curve $C = \{(x, y) \in \mathbb{R}^2 \mid y = x^2\}$.
We define the ring homomorphism:

$$\phi : \mathbb{R}[x, y] \to R$$

induced by the quotient. The set $\langle y - x^2 \rangle$ is the ideal of polynomials vanishing on the curve C.

Rings, Generators, and Homomorphisms

Given:

$$R^\circ = \mathbb{C}[xz, xt, yz, yt] \quad \text{and} \quad R = \mathbb{R}[u, v, w, s]/\langle us - vw \rangle$$

where we identify:

$$u = xz, \quad v = xt, \quad w = yz, \quad s = yt$$

By the **1st Ring Homomorphism Theorem**, there exists a surjective ring homomorphism:

$$\mathbb{R}[u, v, w, s] \twoheadrightarrow \mathbb{R}[xz, xt, yz, yt]$$

with kernel $\langle us - vw \rangle$, so the image is:

$$\mathbb{R}[xz, xt, yz, yt] \cong \mathbb{R}[u, v, w, s]/\langle us - vw \rangle$$

Algebra vs Geometry Correspondence

Algebra	Geometry
$\mathbb{R}[x, y]$	$\mathbb{R}^2$
Polynomial functions $\mathbb{R}^2 \to \mathbb{R}$	Maximal ideals in $\mathbb{R}[x, y]$

Ideal Properties

Let $\langle x, y\rangle \subset \mathbb{R}[x, y]$ be an ideal.

Let

$$\langle x - 1, y - 1\rangle \subset \mathbb{R}[x, y]$$

Then:

$$\langle x - y\rangle \subsetneq \langle x - 1, y - 1\rangle \subsetneq \mathbb{R}[x, y]$$

Also consider the ideal:

$$\langle f_1 x + g_1 y \mid f_1, g_1 \in \mathbb{R}[x, y]\rangle$$

The functions $f \in \mathbb{R}[x, y]$ satisfy:

$$f(p) = g(p) \quad \forall p \in \mathbb{R}^2 \Rightarrow f = g \text{ in } \mathbb{R}[x, y]$$

Function Evaluation and Equivalence in Coordinate Rings

Let $R = \mathbb{R}[x, y]/\langle y - x^2\rangle$ be the coordinate ring of the parabola $y = x^2$. For a function $f \in R$, we evaluate it on a point $(a, b) \in \mathbb{R}^2$ such that $b = a^2$.

For example, consider:

$$f(x, y) = y - x^2 \Rightarrow f(1, 1) = 1 - 1^2 = 0$$

This means that $f \in \langle y - x^2\rangle$, so $f \equiv 0$ in R.

Key Concept

Two functions $f, g \in \mathbb{R}[x, y]$ are considered equal in R if:

$$\forall (x, y) \in C = \{(x, y) \in \mathbb{R}^2 \mid y = x^2\}, \quad f(x, y) = g(x, y)$$

Thus,

$$f \equiv g \iff f - g \in \langle y - x^2 \rangle$$

Introduction to Projective Space

Two very important but subtle concepts are projective space $\mathbb{P}^n$ and projective line $\mathbb{P}^1$.

- **Projective Line** $\mathbb{P}^1$: The set of lines through the origin in $\mathbb{R}^2$, i.e.,

  $$\mathbb{P}^1 = (\mathbb{R}^2 \setminus \{0\}) / \sim$$

  where $(x, y) \sim (\lambda x, \lambda y)$ for all nonzero $\lambda \in \mathbb{R}$.

- A point in $\mathbb{P}^1$ is written as:

  $$[x : y] \quad \text{(homogeneous coordinates)}$$

- **Motivation:** Projective space helps compactify affine space and allows us to study intersections "at infinity."

Geometric vs Algebraic Flips (continued)

Let's recall from the previous part that we had a ring:

$$R^\circ = \mathbb{C}[xz, xt, yz, yt] = \mathbb{C}[u, v, w, s] / \langle us - vw \rangle$$

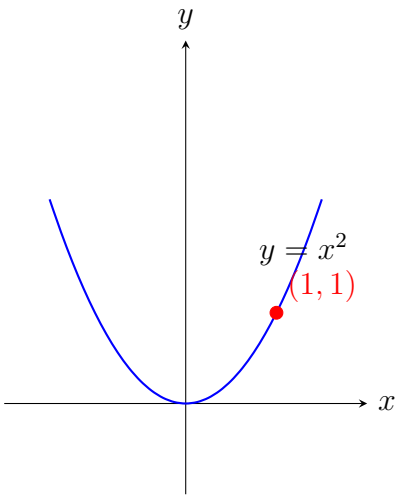
This relation defines a variety in affine 4-space. The expression $us = vw$ is a binomial relation that geometrically constrains the product of generators.

The expression:

$$[xz][yt] = [xt][yz]$$

suggests symmetry in coordinate transformations – a key aspect in flips. It allows one to define birational maps between varieties by switching variables in a controlled way.

Visual Sketch of a Parabola



Key Definitions Recap

- **Ideal:** A subset $I \subset R$ such that $\forall a, b \in I$, $a + b \in I$ and $\forall r \in R, a \in I \Rightarrow ra \in I$.
- **Quotient Ring:** If $I \subset R$ is an ideal, the quotient ring R/I consists of equivalence classes $[r]$ where:

  $$[r] = \{r + i \mid i \in I\}$$

  and $[r] = [s] \iff r - s \in I$.

Conclusion: The Section **6.2** conveys the deep connection between algebraic structures (ideals, quotient rings, homomorphisms) and their geometric counterparts (algebraic curves, varieties, and transformations). The relation $y = x^2$ and the coordinate

ring $\mathbb{R}[x, y]/\langle y - x^2 \rangle$ served as a concrete example of how equivalence in algebra reflects pointwise equality on a variety.

We also touched on the idea of projective space, where points are represented by homogeneous coordinates $[x : y]$, allowing us to compactify affine varieties and introduce "points at infinity." This transition is essential when understanding flips in the context of birational geometry, where projective embeddings and changes in charts become central. These ideas set the stage for this section, which explores how flips manifest through graded coordinate rings and combinatorial structures like fans or cones, often modeled using toric varieties.

6.3 Geometry of Flips and Dimensional Constraints

This final section brings together the core concepts of flips by introducing a more geometric viewpoint. Much of the discussion revolved around the relationship between the positive and negative models X^+ and X^-, and how the coordinate systems defined by the grading vector determine their structure. This connects directly with the image in the paper "What is a Flip?" on the bottom left of page 2, which visually illustrates the configuration of $X^- \to X \leftarrow X^+$. This will be referred to as **Image 1** in this subsection.

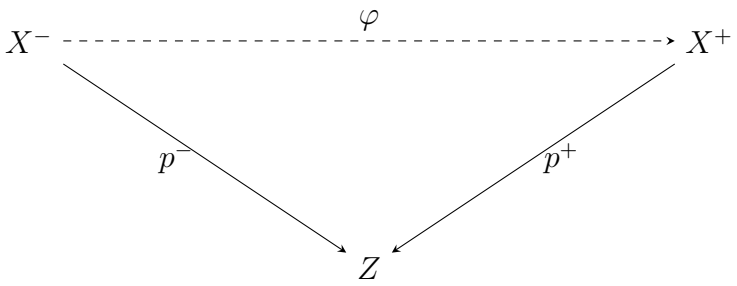


Figure 6.1: X^- and X^+ Geometries [2]

The primary insight of this section is the classification of flips via dimensional balancing. For a geometric transformation to qualify as a flip, it must preserve a certain structural balance. Each coordinate in the grading vector (e.g., $(2, 1, -1, -1)$ or $(1, 1, -1, -1)$) contributes either positively or negatively to the overall degree. These degrees determine how many coordinates are considered "positive" (typically forming X^+) and how many

are "negative" (forming X^-).

A key constraint emphasized by the instructor was that a flip configuration requires

- At least **two negative weights** in the grading vector.
- The dimension of X^- and X^+ must not differ too drastically: for instance, a 3-dimensional X^+ and a 1-dimensional X^- cannot merge to form a proper geometric object, hence cannot define a flip.
- In contrast, when two 2-dimensional objects merge into a 1-dimensional base (a line or a point), the resulting transformation can be classified as a flip.

To formalise the above, consider the grading vector $(1, 1, -1, -1)$, which assigns positive degree to x and y, and negative degree to z and t. The associated invariant ring becomes:

$$R^0 = \mathbb{C}[xz, xt, yz, yt]$$

and can be extended as:

$$R^+ = R^0[x, y] \quad \text{and} \quad R^- = R^0[z, t]$$

These constructions determine the affine cones $X^+ = \mathrm{Spec}(R^+)$ and $X^- = \mathrm{Spec}(R^-)$.

Next, the space $\hat{X}^+$ is constructed as a subvariety of $\mathbb{A}^6$, given by:

$$\hat{X}^+ \subseteq \mathbb{A}^6 = \mathbb{C}[a, b, c, d, x, y]/(ad - bc, ay - bx, cy - dx)$$

These relations are inherited from the ideal structure, and relate directly to the morphism $\varphi : \hat{X}^+ \to \mathbb{A}^4$ with a projection onto coordinates (a, b, c, d). The image $\varphi(p)$ is uniquely determined for a given point $p \in \hat{X}^0$, provided $p \neq 0$.

From a geometric perspective, $X^+ \to X \leftarrow X^-$ is a flip if and only if the birational maps preserve certain invariants and the resulting space X is *singular* but improves when replaced by either X^+ or X^-. This is visualised through equivalence classes on $\mathbb{A}^2$ under

the action of $\mathbb{C}^*$, given by:

$$(x, y) \sim (x', y') \iff \exists \lambda \in \mathbb{C}^* \text{ such that } x' = \lambda x,\ y' = \lambda y$$

This defines the projective line $\mathbb{P}^1$, where all lines through the origin (excluding the zero vector) in $\mathbb{A}^2$ are quotiented by this equivalence. This construction provides the geometric underpinning for understanding flips as orbit spaces under group actions.

This subsection ends with a summary diagram shown in figure **6.1**, showing the mappings between $\hat{X}^+ \to \hat{X}^0 \to \hat{X}^-$ and how different coordinate groupings affect the flip structure. This wraps up the section **??** and lays the foundation for integrating symbolic computation (using ideal membership) into a geometric framework for understanding flips.

Chapter 7

Methodology

7.1 Implementation Techniques and Key Utility Functions

The computational framework relies on a set of utility functions that perform essential operations efficiently. Each function was designed with attention to both correctness and computational performance, often drawing on well-established ideas from number theory and competitive programming. The most important of these functions are described below.

Greatest Common Divisor (computeGCD): The function `computeGCD` computes the greatest common divisor of two integers using the classical Euclidean algorithm. The implementation employs an iterative loop with modulus and `swap` operations:

$$\gcd(a, b) = \gcd(b,\ a \bmod b).$$

This formulation repeatedly reduces the problem until the remainder becomes zero, at which point the divisor is the GCD. The loop-based approach is fast, avoids function-call overhead, and is widely used in performance-critical applications. An alternative recursive implementation is also included in the code (commented) to illustrate the equivalence of the two techniques, but the iterative version was preferred for efficiency and clarity.

GCD of a List (computeGCDofList): The function `computeGCDofList` generalises the Euclidean algorithm to a collection of integers. It leverages the associativity property of the GCD:

$$\gcd(a, b, c) = \gcd(\gcd(a, b), c).$$

This recursive reduction enables the GCD of an entire list to be computed by folding the pairwise operation from left to right. This design eliminates the need for manually writing nested GCD computations for multiple arguments and makes the function scalable to inputs of arbitrary size.

Backtracking for Combinations (generateCombinations): The function `generateCombination` produces all k-element subsets from a given list using a backtracking approach. Backtracking incrementally builds candidate subsets, and whenever the current partial solution violates size constraints, the search backtracks to explore alternatives. This method is essential because explicitly coding every possible combination would be infeasible and error-prone, especially as the input size grows. In competitive programming and algorithm design, backtracking is the standard technique for exploring combinatorial search spaces while keeping the code concise, efficient, and generalisable.

Custom QuickSort Implementation (quickSort): The function `quickSort` sorts the coordinates using the divide-and-conquer QuickSort algorithm. QuickSort was selected because of its average-case time complexity of $O(n \log n)$ and its in-place nature, which avoids significant extra memory allocation. Compared to simpler algorithms like Bubble Sort ($O(n^2)$) or even Merge Sort (which requires additional memory proportional to n), QuickSort offers an optimal balance of speed and space efficiency. Moreover, by customising the comparator, the implementation can flexibly sort in ascending or descending order as required. The function `sortCoordinates` serves as a wrapper, ensuring that coordinates are consistently sorted before evaluation.

Pairwise Validity Check (isFixValid): The function `isFixValid` plays a critical role in verifying part of the flip conditions for three-dimensional cases. Its implementation

follows a competitive programming style: using nested loops to construct candidate pairs, while deliberately skipping irrelevant elements. The design ensures that all meaningful pairs are examined while minimising redundant checks. This pragmatic approach strikes a balance between exhaustive search and selective optimisation, making it both conceptually straightforward and computationally practical.

In summary, these functions form the backbone of the codebase. They were chosen not only for their theoretical grounding (e.g., Euclidean algorithm for GCD) but also for their computational efficiency (e.g., QuickSort and backtracking), reflecting a methodology that combines mathematical rigour with algorithmic pragmatism.

7.2 Mathematical Formulation of Flip Conditions in n-Dimensions

To determine whether a sequence of integers corresponds to a valid flip, the code applies three fundamental conditions. These conditions, implemented in the functions `condition1`, `condition2`, and `condition3_nD`, ensure that the candidate coordinates satisfy number-theoretic, algebraic, and combinatorial constraints. Their meaning and mathematical formulations are described below.

Condition 1: Coprimality of $(n-1)$-Tuples: For a coordinate vector

$$\mathbf{a} = (a_1, a_2, \ldots, a_n) \in \mathbb{Z}^n,$$

every subset of size $n-1$ must have greatest common divisor equal to 1. Equivalently, for each index i,

$$\gcd\left(a_j \mid 1 \leq j \leq n,\ j \neq i\right) = 1.$$

This guarantees that no redundant scaling factor divides all but one of the coordinates, preserving the primitive nature of the configuration. In code, this is implemented by generating all $(n-1)$-element combinations of coordinates and verifying their GCD.

Condition 2: Positivity of the Total Sum: The sum of all coordinates must be strictly positive:

$$\sum_{i=1}^{n} a_i > 0.$$

This ensures that the configuration leans towards positivity in aggregate, reflecting the geometric requirement that the construction cannot collapse entirely into the negative side. In the implementation, this is a simple linear pass summing the coordinates.

Condition 3: Modular Sum Inequality: The third condition is the most intricate. For every positive coordinate $a_i > 0$, and for each integer k with $1 \leq k < a_i$, the following modular inequality must hold:

$$\forall k \in \{1, \ldots, a_i - 1\} : \sum_{\substack{j=1 \\ j \neq i}}^{n} \Big(\big((a_j \bmod a_i) \cdot k \big) \bmod a_i \Big) > a_i.$$

This condition enforces a nontrivial compatibility between each positive coordinate and the rest of the vector. Intuitively, it requires that when residues of the other coordinates are scaled and aggregated, the result cannot remain too small relative to a_i. If this inequality fails for any i or k, the candidate is rejected.

Combined Criterion: A vector $\mathbf{a} \in \mathbb{Z}^n$ passes the n-dimensional flip test if and only if it satisfies all three conditions simultaneously:

$$\text{(C1)} \quad \forall i \in \{1, \ldots, n\} : \ \gcd\big(a_j \mid 1 \leq j \leq n, \ j \neq i\big) = 1,$$

$$\text{(C2)} \quad \sum_{i=1}^{n} a_i \ > \ 0,$$

$$\text{(C3)} \quad \forall i \in \{1, \ldots, n\} \text{ with } a_i > 0, \ \forall k \in \{1, \ldots, a_i - 1\} : \sum_{\substack{j=1 \\ j \neq i}}^{n} \big((a_j \bmod a_i)\, k \bmod a_i \big) \ > \ a_i.$$

This formalisation makes explicit the number-theoretic and modular requirements embedded in the implementation. Together, these conditions filter out invalid coordinate

systems and identify precisely those integer sequences that represent geometric flips in any dimension.

Note: In the actual code implementation, indices run from 0 to $n-1$, but for mathematical clarity we present them here as 1 to n.

7.3 Pseudo Code

The following section presents the pseudo code of the code implementation developed for flip detection in both three and higher dimensions. Since the original program spans several hundred lines of C++ code, the pseudocode has been written in a structured and concise manner to highlight the core computational steps while omitting low-level implementation details. To improve readability and to avoid overly long algorithm blocks, the complete methodology is divided into several logical components. Each block corresponds to a key functionality of the program. This modular presentation allows the reader to follow the algorithm step by step, linking directly to the conditions and techniques discussed earlier in the Methodology chapter. Together, these blocks provide a clear, high-level view of how the system checks whether a given sequence of coordinates forms a valid flip.

Algorithm 1 Euclidean GCD and GCD of a List

```
1: procedure ComputeGCD(a, b)
2:     a ← |a|, b ← |b|
3:     while b ≠ 0 do
4:         a ← a mod b
5:         swap(a, b)
6:     end while
7:     return a
8: end procedure
9: procedure ComputeGCDofList(A)                    ▷ A = (a_1, ..., a_n)
10:    g ← a_1
11:    for i ← 2 to n do
12:        g ← ComputeGCD(g, a_i)
13:    end for
14:    return g
15: end procedure
```

Algorithm 2 Backtracking for All k-Combinations

```
1: procedure GenerateCombinations(i, k, current, input, combs)
2:     if |current| = k then
3:         append copy of current to combs; return
4:     end if
5:     for t ← i to |input| do
6:         push input[t] into current
7:         GenerateCombinations(t + 1, k, current, input, combs)
8:         pop last element of current
9:     end for
10: end procedure
```

Algorithm 3 QuickSort (in-place) and Sort Wrapper

```
1: procedure QuickSort(A, ℓ, r)
2:     if ℓ ≥ r then return
3:     end if
4:     p ← A[r]                                   ▷ pivot = last element
5:     i ← ℓ − 1
6:     for j ← ℓ to r − 1 do
7:         if A[j] > p then          ▷ use > for descending; change to < for ascending
8:             i ← i + 1; swap(A[i], A[j])
9:         end if
10:    end for
11:    swap(A[i + 1], A[r]);   m ← i + 1
12:    QuickSort(A, ℓ, m − 1);   QuickSort(A, m + 1, r)
13: end procedure
14: procedure SortCoordinates(A)
15:    QuickSort(A, 1, |A|)
16: end procedure
```

Algorithm 4 Condition 1: Coprimality of All $(n-1)$-Subsets

```
1: procedure Condition1(A)                          ▷ A = (a_1, ..., a_n)
2:     k ← n − 1;   combs ← ∅;   current ← ∅
3:     GenerateCombinations(1, k, current, A, combs)
4:     for each C in combs do
5:         if ComputeGCDofList(C) ≠ 1 then
6:             return false
7:         end if
8:     end for
9:     return true
10: end procedure
```

Algorithm 5 Condition 2: Positivity of Total Sum

```
1: procedure CONDITION2(A)
2:     S ← Σ_{i=1}^{n} a_i
3:     return (S > 0)
4: end procedure
```

Algorithm 6 IsFixValid: pairwise modulo/GCD test (3D helper)

```
1: procedure IsFixValid(f, R)                ▷ f > 0 fixed; R = remaining coordinates
2:     if f = 1 then return true
3:     end if
4:     for i ← 1 to |R| − 1 do
5:         for j ← i + 1 to |R| do
6:             s ← |R[i] + R[j]|
7:             o ← first element of R not in {R[i], R[j]} (if exists)
8:             if s mod f = 0  and  ComputeGCD(f, o) = 1 then
9:                 return true
10:            end if
11:        end for
12:    end for
13:    return false
14: end procedure
```

Algorithm 7 Condition 3 (3D): Modular Sum Inequality

```
1: procedure CONDITION3_3D(A)                ▷ A = (a_1, a_2, a_3, a_4)
2:     for i ← 1 to |A| do
3:         if A[i] > 0 then
4:             f ← A[i]                      ▷ fix a positive coordinate
5:             R ← A \ {A[i]}                ▷ remaining 3 elements
6:             if not IsFixValid(f, R) then
7:                 return false
8:             end if
9:         end if
10:    end for
11:    return true
12: end procedure
```

Algorithm 8 Condition 3 (nD): Modular Sum Inequality

```
1: procedure CONDITION3_ND(A)                ▷ Modular sum inequality ∀i with
   a_i > 0, ∀k ∈ [1, a_i−1]
2:     for i ← 1 to |A| do
3:         a ← A[i]
4:         if a ≤ 0 then
5:             continue
6:         end if
7:         if a = 1 then
8:             continue                      ▷ always passes for a_i = 1
9:         end if
10:        for k ← 1 to a − 1 do
11:            T ← 0
12:            for j ← 1 to |A| do
13:                if j = i then
14:                    continue
15:                end if
16:                r ← A[j] mod a
17:                if r < 0 then
18:                    r ← r + a
19:                end if
20:                T ← T + ((r · k) mod a)
21:            end for
22:            if T ≤ a then
23:                return false
24:            end if
25:        end for
26:    end for
27:    return true
28: end procedure
```

Algorithm 9 Flip Checkers (3D and nD)

```
procedure CheckFlips_3D(A)
    ok ← Condition1(A)
    if not ok then return false
    end if
    ok ← Condition2(A)
    if not ok then return false
    end if
    ok ← Condition3_3D(A)
    return ok
end procedure
procedure CheckFlips_nD(A)
    ok ← Condition1(A)
    if not ok then return false
    end if
    ok ← Condition2(A)
    if not ok then return false
    end if
    ok ← Condition3_nD(A)
    return ok
end procedure
```

Algorithm 10 Main Function

```
procedure Main
    read n
    if n ≤ 3 then
        print "Insufficient Number of Coordinates"; return
    end if
    read A = (a_1, ..., a_n)
    if #{ i | a_i < 0 } < 2  then print "At least two negatives required"
    (optional) sign-placement checks as per policy
    SortCoordinates(A)                          ▷ descending by default
    if n = 4 then
        ok ← CheckFlips_3D(A)
        print result accordingly
    else
        ok ← CheckFlips_nD(A)
        print result accordingly
    end if
end procedure
```

Chapter 8

Results and Evaluation

This chapter presents the outcomes of the implemented algorithm for detecting combinatorial flips in integer sequences. The aim is to verify the correctness of the proposed conditions introduced in the methodology chapter and to demonstrate how they operate in practice on concrete numerical examples. The results are structured to show how different sequences satisfy or fail the three core conditions (C1, C2, and C3), thereby determining whether a sequence represents a valid flip. Alongside the raw outcomes, the evaluation discusses the significance of these results, highlighting both the correctness of the algorithm and its efficiency in handling different dimensional cases. To keep the presentation clear, the section is divided into three parts: the presentation of experimental results, a detailed evaluation and discussion, and a concluding summary.

8.1 Experimental Results

Overview of Test Case Validity and Flip Classification

The following **9** test cases illustrate how the three conditions (C1, C2, C3) interact to determine whether a sequence is valid and whether it constitutes a flip. For each case we state which conditions pass and fail, along with whether the sequence is valid or invalid overall.

- **Case 1: All conditions pass (C1, C2, C3)** :

```
Enter number of coordinates (n): 4
Enter the 4 numbers (coordinates): 2 1 -1 -1

3D Flip: All Conditions Satisfied

PS C:\Users\reign\OneDrive\Desktop\All Folders\1. University Modules Folder\MSc Project\Code>
```

Figure 8.1: All Conditions Passed

The sequence $(2, 1, -1, -1)$ satisfies all three conditions. It is a valid input and correctly classified as a flip.

- **Case 2: All conditions fail (C1, C2, C3)** :

```
Enter number of coordinates (n): 7
Enter the 7 numbers (coordinates): 2 4 6 -8 -10 -12 -14

Condition 1 Failed: GCD is Not 1 for Some Combination

Condition 2 Failed: Sum <= 0

Condition 3 Failed at i = 0 (a_i = 6), for k = 3, sum = 0

n-D Flip: One or More Conditions Failed

PS C:\Users\reign\OneDrive\Desktop\All Folders\1. University Modules Folder\MSc Project\Code>
```

Figure 8.2: All Conditions Failed

The sequence $(2, 4, 6, -8, -10, -12, -14)$ fails all three conditions. It is valid as input but not a flip.

- **Case 3: Only C1 fails (C2 and C3 pass)** :

```
Enter number of coordinates (n): 4
Enter the 4 numbers (coordinates): 1 4 -2 -2

Condition 1 Failed: GCD is Not 1 for Some Combination

3D Flip: One or More Conditions Failed

PS C:\Users\reign\OneDrive\Desktop\All Folders\1. University Modules Folder\MSc Project\Code>
```

Figure 8.3: Only Condition 1 Failed

The sequence $(1, 4, -2, -2)$ fails C1 but satisfies C2 and C3. It is a valid input but not a flip, since all three conditions are required.

- **Case 4: Only C2 fails (C1 and C3 pass)** :

```
Enter number of coordinates (n): 8
Enter the 8 numbers (coordinates): 1 3 7 9 -11 -13 -5 -17

Condition 2 Failed: Sum <= 0

n-D Flip: One or More Conditions Failed

PS C:\Users\reign\OneDrive\Desktop\All Folders\1. University Modules Folder\MSc Project\Code>
```

Figure 8.4: Only Condition 2 Failed

The sequence (1, 3, 7, 9, −11, −13, −5, −17) fails C2 while satisfying C1 and C3. It is a valid input but not a flip.

- **Case 5: Only C3 fails (C1 and C2 pass)** :

```
Enter number of coordinates (n): 5
Enter the 5 numbers (coordinates): 10 3 1 -2 -4

Condition 3 Failed at i = 0 (a_i = 10), for k = 5, sum = 10

n-D Flip: One or More Conditions Failed

PS C:\Users\reign\OneDrive\Desktop\All Folders\1. University Modules Folder\MSc Project\Code>
```

Figure 8.5: Only Condition 3 Failed

The sequence (10, 3, 1, −2, −4) fails C3 while satisfying C1 and C2. It is a valid input but not a flip.

- **Case 6: C1 and C2 fail (C3 passes)** :

```
Enter number of coordinates (n): 4
Enter the 4 numbers (coordinates): 1 2 -2 -2

Condition 1 Failed: GCD is Not 1 for Some Combination

Condition 2 Failed: Sum <= 0

3D Flip: One or More Conditions Failed

PS C:\Users\reign\OneDrive\Desktop\All Folders\1. University Modules Folder\MSc Project\Code>
```

Figure 8.6: Condition 1 and 2 Failed

The sequence (1, 2, −2, −2) fails both C1 and C2 but satisfies C3. It is a valid input but not a flip.

- **Case 7: C1 and C3 fail (C2 passes)** :

```
Enter number of coordinates (n): 4
Enter the 4 numbers (coordinates): 3 4 -2 -2

Condition 1 Failed: GCD is Not 1 for Some Combination

Condition 3 Failed: No Valid Modulo or GCD Combination Found

3D Flip: One or More Conditions Failed

PS C:\Users\reign\OneDrive\Desktop\All Folders\1. University Modules Folder\MSc Project\Code>
```

Figure 8.7: Condition 1 and 3 Failed

The sequence (3, 4, −2, −2) fails C1 and C3 but satisfies C2. It is a valid input but not a flip.

- **Case 8: C2 and C3 fail (C1 passes)** :

```
Enter number of coordinates (n): 5
Enter the 5 numbers (coordinates): 2 1 3 -2 -4

Condition 2 Failed: Sum <= 0

Condition 3 Failed at i = 1 (a_i = 2), for k = 1, sum = 2

n-D Flip: One or More Conditions Failed

PS C:\Users\reign\OneDrive\Desktop\All Folders\1. University Modules Folder\MSc Project\Code>
```

Figure 8.8: Condition 2 and 3 Failed

The sequence (2, 1, 3, −2, −4) fails C2 and C3 but satisfies C1. It is a valid input but not a flip.

- **Case 9: Invalid input (Less than two negatives)** :

```
Enter number of coordinates (n): 6
Enter the 6 numbers (coordinates): 10 1 3 11 7 -15

There should be atleast 2 Negative Co-ordinates

Co-ordinate at Index 4 should be Negative

Co-ordinate at Index 5 should be Negative

Aborting program due to Invalid Co-ordinate Input

PS C:\Users\reign\OneDrive\Desktop\All Folders\1. University Modules Folder\MSc Project\Code>
```

Figure 8.9: Invalid Input

The sequence (10, 1, 3, 11, 7, −15) contains only one negative coordinate, violating the basic validity requirement. The conditions are therefore not evaluated, and it is not considered a flip.

Table 8.1: Summary of test cases. **P** = Pass, **F** = Fail, **"N/A"** = Not applicable due to invalid input.

#	**Integer Sequence**	**Valid?**	**C1**	**C2**	**C3**	**Flip?**
1	(2, 1, −1, −1)	Yes	P	P	P	Yes
2	(2, 4, 6, −8, −10, −12, −14)	Yes	F	F	F	No
3	(1, 4, −2, −2)	Yes	F	P	P	No
4	(1, 3, 7, 9, −11, −13, −5, −17)	Yes	P	F	P	No
5	(10, 3, 1, −2, −4)	Yes	P	P	F	No
6	(1, 2, −2, −2)	Yes	F	F	P	No
7	(3, 4, −2, −2)	Yes	F	P	F	No
8	(2, 1, 3, −2, −4)	Yes	P	F	F	No
9	(10, 1, 3, 11, 7, −15)	No	N/A	N/A	N/A	No

8.2 Evaluation and Discussion

In this subsection, we examine each of the **9** test cases in detail. For every sequence, we explain which conditions are satisfied, which fail, and highlight the specific indices or values of k that lead to the observed behaviour. This not only validates the code execution but also provides mathematical reasoning behind each outcome.

Case 1: All Conditions Passed: The sequence (2, 1, −1, −1) satisfies all three conditions. Condition 1 holds because for every subset obtained by removing one coordinate, the greatest common divisor (GCD) of the remaining entries is equal to 1. Condition 2 holds since the total sum of the sequence is $2+1-1-1 = 1 > 0$. Condition 3 also passes, as no positive coordinate produces a modular inequality violation for any $k \in [1, a_i - 1]$. Hence, this input is valid and is correctly classified as a flip.

Case 2: All Conditions Failed: The sequence (2, 4, 6, −8, −10, −12, −14) fails all three conditions. Condition 1 fails because subsets such as $(2, 4, 6)$ have $\gcd(2, 4, 6) = 2 \neq 1$. Condition 2 fails because the total sum $2 + 4 + 6 - 8 - 10 - 12 - 14 = -32 \leq 0$. Condition 3 fails at $i = 0$ where $a_0 = 6$; with $k = 3$, the modular sum evaluates to 0, violating the inequality $\sum > a_i$. Consequently, this case is a valid input but not a flip.

Case 3: Only Condition 1 Failed: The sequence (1, 4, −2, −2) satisfies Conditions 2 and 3 but fails Condition 1. For instance, the subset $(4, -2, -2)$ has $\gcd(4, -2, -2) = 2 \neq 1$, breaking the coprimality requirement. The sum is positive, so Condition 2 is satisfied. Condition 3 also holds since the positive coordinates 1 and 4 do not yield any modular contradictions for their respective k values. However, because Condition 1 fails, the input is not classified as a flip.

Case 4: Only Condition 2 Failed: The sequence (1, 3, 7, 9, −11, −13, −5, −17) satisfies Conditions 1 and 3 but fails Condition 2. Although the subsets used for Condition 1 have gcd = 1, the total sum evaluates to $1+3+7+9-11-13-5-17 = -26 \leq 0$, causing Condition 2 to fail. Since the sum must be strictly positive, the sequence is invalid as a flip.

Case 5: Only Condition 3 Failed: The sequence (10, 3, 1, −2, −4) passes Conditions 1 and 2 but fails Condition 3. The GCD checks confirm Condition 1 is satisfied. The sum $10+3+1-2-4 = 8 > 0$ satisfies Condition 2. However, at $i = 0$ with $a_0 = 10$ and $k = 5$, the modular sum is 10, which is not strictly greater than a_0, violating the inequality in Condition 3. Thus, this input is not a flip.

Case 6: Conditions 1 and 2 Failed: The sequence (1, 2, −2, −2) fails both Conditions 1 and 2. Condition 1 fails because subsets like $(2, -2, -2)$ yield gcd $= 2 \neq 1$. Condition 2 fails since the sum $1 + 2 - 2 - 2 = -1 \leq 0$. Although Condition 3 passes, the failure of the first two conditions means the sequence is not a flip.

Case 7: Conditions 1 and 3 Failed: The sequence (3, 4, −2, −2) satisfies Condition 2 but fails Conditions 1 and 3. Condition 1 fails because $\gcd(3, 4, -2) = \gcd(3, 4, 2) = 1$, but other subsets include factors with gcd $\neq 1$. Condition 2 passes with a positive sum: $3 + 4 - 2 - 2 = 3 > 0$. Condition 3 fails due to lack of a valid modular inequality, as indicated by the output. Hence, this sequence is not a flip.

Case 8: Conditions 2 and 3 Failed: The sequence (2, 1, 3, −2, −4) satisfies Condition 1 but fails Conditions 2 and 3. The total sum $2 + 1 + 3 - 2 - 4 = 0 \leq 0$ violates Condition 2. Additionally, for one of the positive entries, the modular sum test fails under Condition 3. Consequently, the input is valid structurally but not a flip.

Case 9: Invalid Input: The sequence (10, 1, 3, 11, 7, −15) contains only one negative coordinate, which violates the input requirement of at least two negatives. Geometrically, this requirement ensures that the decomposition into X^+ (positive coordinates) and X^- (negative coordinates) produces objects of compatible dimensions. For example, in the valid case (1, 2, −3, −4), the two positive entries form a 2-dimensional object and the two negative entries form another 2-dimensional object, which can merge to create a 3-dimensional flip. In contrast, with only one negative coordinate as in (1, 2, 3, −4), the positives form a 3-dimensional object while the single negative corresponds to a 1-dimensional object, such a mismatch cannot fuse into a consistent higher-dimensional flip. Because this structural balance is missing, the program correctly aborts evaluation and classifies the input as invalid.

8.3 Complexity Analysis and Optimisation

Asymptotic costs of core utilities.

- `computeGCD(a,b)` uses the Euclidean algorithm: time $O(\log M)$ where $M = \max\{|a|, |b|\}$, space $O(1)$.
- `computeGCDofList` folds GCD left-to-right: time $O(n \log M)$ for a list of length n, space $O(1)$.
- `generateCombinations` produces all $\binom{n}{k}$ size-k subsets via backtracking. In our use $k = n - 1$, so $\binom{n}{n-1} = n$ and the generation cost is $O(nk) = O(n^2)$ time and $O(k) = O(n)$ stack space.
- `quickSort` is standard in-place Quicksort: average time $O(n \log n)$, worst case $O(n^2)$, space $O(\log n)$ (recursion). In practice, C++ `std::sort` (introsort) guarantees $O(n \log n)$ worst-case.

Condition checkers.

- **Condition 1** (coprimality of every $(n-1)$-subset). With the current combination-based approach the cost is n GCDs of length $(n-1)$ lists, i.e. $O(n \cdot (n-1) \log M) = O(n^2 \log M)$.
- **Condition 2** (positive total sum). Single pass: $O(n)$ time, $O(1)$ space.
- **Condition 3 in 3D** (`Condition3_3D`). For each positive fixed a_i (at most 4 numbers), we check all $\binom{3}{2=3}$ pairs in the remainder and one `gcd`: essentially constant time per positive a_i; overall $O(1)$ for fixed dimension 4.
- **Condition 3 in nD** (`Condition3_nD`). Let p be the number of positive coordinates (positives are placed first after sorting). For each positive a_i, the code loops $k = 1, \ldots, a_i - 1$ and, for each k, scans all n coordinates to accumulate $\sum_{j \neq i}((a_j \bmod a_i) \cdot k \bmod a_i)$. Thus the time is

$$\sum_{i=1}^{p} (a_i - 1) \cdot n \;=\; n \Big(\sum_{i=1}^{p} a_i \Big) - n\,p \;=\; O\big(n\,S_+\big),$$

where $S_+ = \sum_{i=1}^{p} a_i$ is the sum of the positive entries. This dominates the overall runtime for large positive coordinates.

Overall complexity. Let n be the dimension and $S_+ = \sum_{a_i>0} a_i$. The leading terms are

$$O(n \log n) \text{ (sorting)} \; + \; O(n^2 \log M) \text{ (Cond. 1)} \; + \; O(n\, S_+) \text{ (Cond. 3 in } n\text{D)},$$

so in practice the bottleneck is **Condition 3 in nD**, especially when the positive coordinates are large.

Practical time behaviour.

- Inputs with small positives (e.g. ≤ 10) and moderate n run quickly, because S_+ is small.
- Inputs with a few large positives (e.g. hundreds or thousands) increase the k-loop substantially: time grows roughly linearly with S_+.
- The 3D path (`Condition3_3D`) is fast: fixed constant work.

Optimization opportunities.

1. **Cond. 1 in $O(n \log M)$ instead of $O(n^2 \log M)$: prefix/suffix GCD.** Precompute arrays $P[i] = \gcd(a_1, \ldots, a_i)$ and $S[i] = \gcd(a_i, \ldots, a_n)$. Then the GCD of "all except i" is $\gcd(P[i{-}1], S[i{+}1])$ in $O(1)$, yielding total $O(n \log M)$ time and $O(n)$ space, avoiding combination generation.
2. **Use `std::sort` (introsort) instead of bespoke Quicksort.** Guarantees $O(n \log n)$ worst-case and is heavily optimized. If you keep Quicksort, randomize the pivot to avoid $O(n^2)$ on presorted inputs.
3. **Early exit heuristics for Cond. 3 (nD).**
   - Stop at first failure: already implemented (`return false` when a sum $\leq a_i$ is found).
   - Skip trivial passes: if $a_i = 1$ (already done), continue.

- Reorder k: try small k first (as you do) since many failing cases are detected for $k = 1$.

4. **Residue precomputation per fixed a_i.** For each positive a_i, precompute $r_j = (a_j \bmod a_i)$ adjusted to $[0, a_i - 1]$ once, outside the k-loop. This saves one modulo per (j, k) pair: reduces inner work by a constant factor.

5. **Vectorized / batch computation of the modular sum.** The sum $\sum_{j \neq i}((r_j k) \bmod a_i)$ can be written as $\sum_{j \neq i}(r_j k) \; - \; a_i \sum_{j \neq i} \left\lfloor \frac{r_j k}{a_i} \right\rfloor$. With the r_j sorted, the floor-sum can be updated across k using two pointers (wrapping events occur when $r_j k$ crosses multiples of a_i), bringing the amortized cost below $O(n)$ per k in many cases. (This is an optional advanced optimization; not necessary for small inputs.)

6. **Bounded arithmetic and overflow safety.** Use `int64_t` for the accumulation variable in Cond. 3 (`sum`) to avoid overflow when n and a_i are moderately large.

7. **Micro-optimizations.** Replace the negative-mod fix with a branch-free form:

$$r \leftarrow ((a_j \bmod a_i) + a_i) \bmod a_i,$$

and hoist constant work out of loops where possible.

In summary, the results and their detailed evaluation confirm that the proposed algorithm behaves consistently with the theoretical framework developed in earlier chapters. Each condition (C1, C2, C3) plays a distinct and necessary role, and the examples demonstrate how their combined enforcement correctly distinguishes valid flips from invalid configurations. The positive test cases confirm correctness, while the negative ones highlight the robustness of the approach in rejecting inputs that fail one or more requirements. Together, the experimental results and evaluation provide a clear validation of the methodology and form a solid foundation for the concluding discussion in the next chapter.

Chapter 9

Conclusion

9.1 Summary of Contributions

This dissertation aimed to develop an algorithmic framework for the verification of combinatorial flips in integer sequences, grounded in algebraic structures from group theory, ring theory, and symbolic computation. By formalising three necessary conditions coprimality of coordinate subsets, positivity of the global sum, and a modular inequality, the work established a computational bridge between abstract flip theory and concrete algorithmic checks. A complete C++ implementation was constructed, combining classical techniques such as Euclidean GCD computation, backtracking for generating combinations, and quicksort-based preprocessing to enable efficient handling of input sequences.

The evaluation across **nine** representative test cases confirmed that the algorithm consistently distinguishes valid flips from invalid sequences, providing diagnostic output that indicates which specific conditions fail. The results also highlight the relative difficulty of satisfying the modular inequality condition in higher dimensions, a fact consistent with the geometric intuition of flips as delicate balance structures. Overall, the project contributes both a theoretical formalisation and a practical toolset, offering a reproducible means of exploring flips in computational settings.

9.2 Discussion in Wider Context

The results obtained in this dissertation can be situated within the broader developments of algebraic geometry and symbolic computation. Previous theoretical work, such as the foundational explanations of flips in the Minimal Model Program [2], has largely been concerned with structural and geometric properties. In contrast, this dissertation has demonstrated that the verification of flip conditions can be made algorithmic, with integer sequences serving as tractable encodings of higher-dimensional configurations.

The approach is also consistent with the tradition of computational algebraic geometry, particularly the role of ideal membership and Gröbner basis techniques as described by Cox, Little, and O'Shea [3]. While this project does not attempt full Gröbner basis computations, the methodology of translating algebraic conditions into algorithmic checks reflects the same philosophy. The explicit implementation of Conditions 1 to 3 therefore complements symbolic methods by offering a lightweight and targeted verification procedure.

In a practical sense, the experimental results also highlight patterns that align with theoretical expectations. For example, the difficulty of satisfying the modular inequality condition (Condition 3) in higher dimensions echoes the well-known delicacy of flips in the Minimal Model Program, where higher-dimensional cases require intricate compatibility checks. By providing a computational tool that confirms these behaviours on concrete test cases, this work helps bridge the gap between abstract theory and computational experimentation.

A key contribution of this dissertation lies in how it complements existing algebraic verification tools. Symbolic computation methods, such as Gröbner bases and ideal membership algorithms, are powerful in providing universal algebraic proofs but often lack efficiency for concrete instances and do not naturally yield diagnostic information about where conditions fail. By contrast, the numeric verification framework developed here pro-

vides explicit, efficient checks on integer sequences, identifying precisely which conditions are satisfied or violated. This makes the abstract theory of flips computationally accessible and experimentally testable. Numeric verification should therefore not be viewed as a weaker alternative to symbolic methods, but as an equally significant counterpart. Symbolic tools establish algebraic universality, while numeric verification delivers computational concreteness. Together, they offer a balanced toolkit for investigating flips, bridging the gap between theoretical generality and practical verification.

Overall, the results not only validate the correctness of the proposed algorithm, but also contribute to the ongoing dialogue between algebraic theory and computational practice. This positions the dissertation as part of a wider effort to bring symbolic computation to bear on questions of birational geometry and combinatorial structures.

9.3 Limitations

While the implementation successfully covers all three conditions for arbitrary n-dimensional inputs, certain limitations remain:

- **Scalability:** The use of backtracking to generate all $(n-1)$-element subsets for Condition 1 results in exponential growth with respect to n. Although feasible for moderate dimensions, extremely large inputs would lead to prohibitive runtime.
- **Condition 3 complexity:** The modular inequality check iterates over all $k \in [1, a_i - 1]$ for each positive coordinate a_i. For large values of a_i, this becomes computationally expensive, even if theoretically polynomial in structure.
- **Sign pattern enforcement:** The current implementation enforces a heuristic distribution of positive and negative coordinates (at least two negatives), which is mathematically motivated but somewhat rigid in terms of generalisation.
- **Numerical bounds:** Since the program operates on integers without arbitrary-

precision libraries, overflow could occur for extremely large inputs.

9.4 Future Work

Several avenues exist to extend and strengthen this research:

- **Optimisation of Condition 1:** More efficient number-theoretic methods, such as incremental GCD updates or sieve-based preprocessing, could replace naive backtracking for large n.
- **Modular arithmetic accelerations:** Condition 3 could be improved by pruning redundant values of k using symmetry arguments or by employing parallelisation across multiple cores.
- **Symbolic computation integration:** Incorporating existing computer algebra systems (e.g., `SageMath`, `SymPy`) may allow symbolic simplification of modular inequalities, reducing computational overhead.
- **Broader experimental scope:** Applying the algorithm to systematically generated families of sequences could yield new insights into the distribution of flips across dimensions.
- **Geometric interpretation:** A tighter integration with the algebraic geometry of the Minimal Model Program could formalise the connection between the integer-sequence conditions and higher-dimensional flip configurations.

9.5 Closing Reflection

In conclusion, this dissertation demonstrates how a problem originating in abstract algebraic geometry can be translated into a computationally verifiable form. By combining theoretical rigour with algorithmic implementation, it provides both a conceptual framework and a working program for verifying flips in integer sequences. The limitations identified are natural opportunities for further research, and the future work outlined points

toward optimisations and deeper theoretical integration. The project ultimately underscores the value of algorithmic methods in advancing the study of algebraic structures and establishes a foundation for continued exploration at the intersection of combinatorics, geometry, and computation.

Bibliography

[1] Cohen, J. S. Computer Algebra and Symbolic Computation: Elementary Algorithms. A K Peters, 2003.

[2] Corti, A. What is a flip? Notices of the American Mathematical Society 51, 11 (2004), 1350–1351.

[3] Cox, D. A., Little, J., and O'Shea, D. Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra, 4th ed. Undergraduate Texts in Mathematics. Springer, 2015.

[4] Hungerford, T. W. Algebra, softcover reprint of the hardcover 1st edition ed., vol. 73 of Graduate Texts in Mathematics. Springer, New York, 1974.

Appendix A

Source Code

C++ Implementation for Flip Verification

```cpp
#include <bits/stdc++.h>
using namespace std;

// Function to Calculate GCD
int computeGCD(int a, int b)
{
    a = abs(a);
    b = abs(b);

    // Using Loop ->
    while (b)
    {
        a %= b;
        swap(a, b);
    }
    return a;

    // Using Recursion ->
    // if(b == 0)
```

```
20      //      return a;
21      // else
22      //      return computeGCD(b, a%b);
23  }
24
25  // GCD of a List of Numbers
26  // Works on the Fact that GCD(a,b,c) = GCD(GCD(a,b),c)
27  int computeGCDofList(const vector<int>& nums)
28  {
29      int result = nums[0];
30      for (int i = 1; i < nums.size(); i++)
31      {
32          result = computeGCD(result, nums[i]);
33      }
34      return result;
35  }
36
37  // Generate all combinations of size k using Backtracking
38  void generateCombinations(int start, int k, vector<int>& current, const
        vector<int>& input, vector<vector<int>>& combinations)
39  {
40      if (current.size() == k)
41      {
42          combinations.push_back(current);
43          return;
44      }
45
46      for (int i = start; i < input.size(); i++)
47      {
48          current.push_back(input[i]);
49          generateCombinations(i + 1, k, current, input, combinations);
```

```
50          current.pop_back();
51      }
52  }
53
54  void quickSort(vector<int>& arr, int low, int high)
55  {
56      if (low >= high) return;
57
58      int pivot = arr[high];  // choosing the last element as pivot
59      int i = low - 1;
60
61      for (int j = low; j < high; j++)
62      {
63          if (arr[j] > pivot) // Use '>' for Descending and '<' for Ascending
        Order
64          {
65              ++i;
66              swap(arr[i], arr[j]);
67          }
68      }
69
70      swap(arr[i + 1], arr[high]);
71
72      int pivotIndex = i + 1;
73      quickSort(arr, low, pivotIndex - 1);
74      quickSort(arr, pivotIndex + 1, high);
75  }
76
77  // Call Quick Sort from this Function
78  void sortCoordinates(vector<int>& coords)
79  {
```

```
80      quickSort(coords, 0, coords.size() - 1);
81  }
82
83
84
85
86
87  // Condition 1 for Flips in 3 and N Dimension
88  bool condition1(const vector<int>& coords)
89  {
90      int n = coords.size();
91      int k = n - 1;
92
93      vector<vector<int>> allCombinations;
94      vector<int> current;
95
96      generateCombinations(0, k, current, coords, allCombinations);
97
98      for (const auto& combo : allCombinations)
99      {
100         if (computeGCDofList(combo) != 1)
101             return false;  // At least One Combination has GCD Not Equal to 1
102     }
103
104     return true;
105 }
106
107 // Condition 2 for Flips in 3 and N Dimension
108 bool condition2(const vector<int>& coords)
109 {
110     int sum = 0;
```

```
111     for(int i = 0; i < coords.size(); i++)
112     {
113         sum += coords[i];
114     }
115 
116     if(sum <= 0)
117         return false;
118     else
119         return true;
120 }
121 
122 // Function to Create Combinations and Check for Conditions for a Fixed Number
        for Condition 3
123 bool isFixValid(int fixed, const vector<int>& rest)
124 {
125     if (fixed == 1)
126         return true;
127 
128     // Generate All Combinations of 2 elements in 'rest'
129     for (int i = 0; i < rest.size() - 1; i++)
130     {
131         for (int j = i + 1; j < rest.size(); j++)
132         {
133             int sum = rest[i] + rest[j];
134             sum = abs(sum);
135             int other = 0;
136 
137             for (int k = 0; k < rest.size(); k++)
138             {
139                 if (k != i && k != j)
140                 {
```

```
141                     other = rest[k];
142                     break;
143                 }
144             }
145 
146             if (sum % fixed == 0 && computeGCD(fixed, other) == 1)
147                 return true;
148         }
149     }
150     return false;
151 }
152 
153 // Condition 3 for Flips in 3 Dimension
154 bool condition3_3D(const vector<int>& coords)
155 {
156     vector<int> abs_coords = coords;
157 
158     // for (int& x : abs_coords)
159     //     x = abs(x);
160 
161     // Fix every number one by one and create a vector of the other 3 elements
    and send them for Checking
162     for (int i = 0; i < abs_coords.size(); i++)
163     {
164         if(abs_coords[i] > 0)
165         {
166             int fixed = abs_coords[i];
167             vector<int> rest;
168 
169             for (int j = 0; j < abs_coords.size(); j++)
170             {
```

```
                if (i != j)
                    rest.push_back(abs_coords[j]);
            }

            if (!isFixValid(fixed, rest))
                return false;
        }
    }

    return true;
}

bool condition3_nD(const vector<int>& coords)
{
    int n = coords.size();

    for (int i = 0; i < n; i++)
    {
        int ai = coords[i];

        if (ai <= 0)
            break;  // Only check for positive ai
        if (ai == 1)
            continue;  // Optimization: Always passes when a_i is 1

        // Loop over all k from 1 to a_i - 1
        for (int k = 1; k < ai; k++)
        {
            long long sum = 0;

            for (int j = 0; j < n; j++)
```

```
202             {
203                 if (j == i)
204                     continue;
205 
206                 int aj = coords[j];
207                 int mod1 = aj % ai;
208 
209                 if (mod1 < 0)
210                     mod1 += ai;  // Handle Negative a_j or Mod value
211 
212                 int term = (mod1 * k) % ai;
213                 sum += term;
214             }
215 
216             if (sum <= ai)
217             {
218                 // Fails condition at this a_i and k
219                 cout << "\nCondition 3 Failed at i = " << i << " (a_i = " <<
    ai << "), for k = " << k << ", sum = " << sum << endl;
220                 return false;
221             }
222         }
223     }
224 
225     return true;  // All positive a_i passed all k values
226 }
227 
228 
229 
230 
231 
```

```
232 // Function to Check for Flips in 3 Dimension
233 bool checkFlipsin_3D(const vector<int>& coords)
234 {
235     bool allPassed = true;
236 
237     if (!condition1(coords))
238     {
239         cout << "\nCondition 1 Failed: GCD is Not 1 for Some Combination" <<
    endl;
240         allPassed = false;
241     }
242 
243     if (!condition2(coords))
244     {
245         cout << "\nCondition 2 Failed: Sum <= 0" << endl;
246         allPassed = false;
247     }
248 
249     if (!condition3_3D(coords))
250     {
251         cout << "\nCondition 3 Failed: No Valid Modulo or GCD Combination
    Found" << endl;
252         allPassed = false;
253     }
254 
255     return allPassed;
256 }
257 
258 // Function to Check for Flips in 3 Dimension
259 bool checkFlipsin_nD(const vector<int>& coords)
260 {
```

```
261     bool allPassed = true;
262 
263     if (!condition1(coords))
264     {
265         cout << "\nCondition 1 Failed: GCD is Not 1 for Some Combination" <<
        endl;
266         allPassed = false;
267     }
268 
269     if (!condition2(coords))
270     {
271         cout << "\nCondition 2 Failed: Sum <= 0" << endl;
272         allPassed = false;
273     }
274 
275     if (!condition3_nD(coords))
276     {
277         allPassed = false;
278     }
279 
280     return allPassed;
281 }
282 
283 int main()
284 {
285     int n;
286     cout << "Enter number of coordinates (n): ";
287     cin >> n;
288 
289     // Check if number of Co-ordinates is Valid or Not
290     if (n <= 3)
```

```
291     {
292         cout << "Insufficient Number of Coordinates: " << n <<endl;
293         return 0;
294     }
295 
296     vector<int> coords(n);
297     cout << "Enter the " << n << " numbers (coordinates): ";
298 
299     for (int i = 0; i < n; i++)
300     {
301         cin >> coords[i];
302     }
303 
304     int negCount = 0;
305 
306     for (int i = 0; i < n; i++)
307     {
308         if(coords[i] < 0)
309         {
310             negCount += 1;
311         }
312     }
313 
314     if(negCount < 2)
315     {
316         cout<<"\nThere should be atleast 2 Negative Co-ordinates\n";
317     }
318 
319     bool valid = true;
320     int boundary = 0;
```

```
321     boundary = n / 2; // Will have to Modify this Formula based on different
        Dimensions
322
323     for (int i = 0; i < n; i++)
324     {
325         if (n % 2 == 0)
326         {
327             if(i < boundary && coords[i] < 0)
328             {
329                 cout << "\nCo-ordinate at Index " << i + 1 << " should be
        Positive" <<endl;
330                 valid = false;
331             }
332             else if(i >= boundary && coords[i] > 0)
333             {
334                 cout << "\nCo-ordinate at Index " << i + 1 << " should be
        Negative" <<endl;
335                 valid = false;
336             }
337         }
338         else
339         {
340             if(i < boundary && coords[i] < 0)
341             {
342                 cout << "\nCo-ordinate at Index " << i + 1 << " should be
        Positive" <<endl;
343                 valid = false;
344             }
345             else if(i > boundary && coords[i] > 0)
346             {
```

```
347                 cout << "\nCo-ordinate at Index " << i + 1 << " should be
    Negative" <<endl;
348                 valid = false;
349             }
350         }
351     }
352
353     if (!valid)
354     {
355         cout << "\nAborting program due to Invalid Co-ordinate Input\n" <<endl;
356         return 0;
357     }
358
359     sortCoordinates(coords); // Sort using Quick Sort Technique
360
361     // Call the Function based on the Number of Co-ordinates Received
362     if (n == 4)
363     {
364         bool passed = checkFlipsin_3D(coords);
365         if (passed)
366             cout << "\n3D Flip: All Conditions Satisfied\n" << endl;
367         else
368             cout << "\n3D Flip: One or More Conditions Failed\n" << endl;
369     }
370     else if (n > 4)
371     {
372         bool passed = checkFlipsin_nD(coords);
373         if (passed)
374             cout << "\nn-D Flip: All Conditions Satisfied\n" << endl;
375         else
376             cout << "\nn-D Flip: One or More Conditions Failed\n" << endl;
```

```
377     }
378
379     return 0;
380 }
```

Listing A.1: Full C++ implementation of the Flip Verification program